

Shandess

Adapting Large Language Models for Student Response Prediction

Adèle Dejoie

Abstract

Predicting student responses is a key component of adaptive exercise selection. Pretrained language models can exploit exercise content, but representing long interaction histories as text consumes their context window and increases inference cost. This report investigates how to encode these histories compactly and condition a shared language model on individual student experience. We introduce Shandess, a framework for knowledge tracing and multiple-choice answer prediction that combines precomputed semantic exercise embeddings with interaction-specific information through a dedicated History Encoder. We compare two conditioning mechanisms: Shandess Soft Prompt, which prepends continuous history representations to the model input, and Shandess HiCA, which introduces gated History Cross-Attention between decoder layers. Experiments on the MIAAM mathematics dataset use Qwen2.5-0.5B-Instruct and examine history representation, architectural variants, and training distributions. On a common knowledge-tracing test of 4,000 interactions from held-out students, Shandess HiCA h1 achieves 78.375% accuracy and Shandess Soft Prompt h1 with AM/MIA 50/50 adaptation achieves 77.625%, compared with 75.350% for a history-independent language-model control and 75.975% for Deep Knowledge Tracing. Here h1 denotes a minimum training history of one interaction. Evaluations at fixed prediction targets show benefits from longer histories and greater robustness of Shandess HiCA to very short contexts. A small evaluation on unseen exercises provides preliminary evidence of generalization within MIAAM. Shandess thus offers a practical way to combine exercise semantics and student histories in a predictor with fewer than one billion parameters, while recurrent baselines retain substantially lower inference latency.

Internship context and acknowledgements

This work was carried out during my MVA research internship in the Flowers team at Inria, under the supervision of Pierre-Yves Oudeyer and Loris Gaven. I would like to thank them for their guidance and feedback throughout the internship, as well as Clément Romac for his advice and the fruitful discussions that helped shape this work.

Contents

1	Introduction	3
2	Intended Use, Model Requirements, and Prediction Tasks	4
2.1	From response prediction to personalized exercise recommendation	4
2.2	Requirements for large-scale deployment	5
2.3	Why study both KT and AP?	5
3	Background and Related Work	6
3.1	Classical and structured knowledge tracing	6
3.2	Knowledge tracing from exercise content	7
3.3	Language models for competence and learning-progress prediction	9
3.4	Comparison with the intended-use requirements	10
4	MIAAM	11
4.1	MIAAM dataset	11
4.2	Dataset distributions relevant to student modeling	12
4.3	Prediction metrics	14
5	Designing Shandess	14
5.1	Using a language model as the prediction model	14
5.2	Separating the history from the text prompt	16
5.3	Injecting history tokens into the language model	17
5.4	Improving the History Encoder	20
5.5	Final Shandess architecture	23
6	Training Shandess	25
6.1	Dataset splitting and prediction windows	25
6.2	KT and AP objectives	26
6.3	Target sampling and checkpoint selection	26
6.4	History thresholds and the evaluated models	27
6.5	Initialization and computational setup	27
7	Final Evaluation and Baseline Comparison	28
7.1	Knowledge Tracing: predictive gain over the baselines	28
7.2	Inference cost of the predictive gain	29
7.3	Quality of the predicted probabilities	30
7.4	New students and limited history	31
7.5	Prediction on new exercises	33
7.6	Answer Prediction: predicting student errors	34
7.7	Does Shandess meet the intended-use requirements?	34
8	Future Work	34
8.1	Prediction model family, size and computational cost	34

8.2	Cross-dataset generalization and rapid adaptation	35
8.3	From student prediction to personalized trajectory construction	36
9	Conclusion	36
A	Training details and supplementary experiments	39
A.1	Window populations	39
A.2	Correctness sampling in Answer Prediction	39
A.3	Balancing Adaptiv’Math and MIA Seconde	40
A.4	Initializations and KT training trajectories	41
A.5	Computational setup and costs	43
B	Complete evaluation results	44
B.1	Knowledge Tracing	44
B.2	Answer Prediction	46
B.3	Probability quality	48
B.4	Latency and model constraints	48
C	Uncertainty of the KT accuracy differences	50
D	Validation runs and checkpoints	51
D.1	Validation cache identities	51
D.2	Main KT configurations and intermediate phases	52
D.3	Shandess Soft Prompt h1 runs	53
D.4	Shandess HiCA h1 checkpoint record	53
D.5	Other historical protocols	53
D.6	AP configurations	54
E	Cold-start KT details and uncertainty	54
E.1	Naturally available histories	54
E.2	Fixed-target history sweep	55
E.3	First encounters with test-only exercises	57
E.4	Uncertainty of the cold-start comparisons	57
F	Task instructions and illustrative input	58
F.1	Knowledge Tracing system instruction	58
F.2	Answer Prediction system instruction	58
F.3	Schematic example	59

1 Introduction

Adaptive learning systems seek to select exercises that are appropriate for each student. To make such decisions, they need predictive models that estimate how a student is likely to respond to a given exercise. In Knowledge Tracing (KT), the objective is to predict whether the student will answer the exercise correctly, based on their previous interactions. Answer Prediction (AP) considers the related problem of predicting the exact answer selected by the student. These predictions do not directly define a personalized learning trajectory. However, they can be used by a separate exercise-selection mechanism to compare candidate activities and construct a trajectory adapted to the student’s current abilities. Improving KT and AP models can therefore contribute to more relevant exercise selection and, ultimately, to more effective learning.

Most existing KT models represent exercises through discrete identifiers, sometimes supplemented with metadata such as the associated skill, module, or exercise type. This approach can perform well when the set of exercises is fixed, but remains rigid. A new exercise has no learned representation and must first be manually assigned the expected metadata. The model may then need to be updated or retrained before the exercise can be properly integrated. Moreover, two semantically related exercises are not automatically recognized as such unless this relation is explicitly encoded in their metadata or can be inferred from sufficient interaction data. This limitation partly reflects the structure of commonly used KT datasets, which often provide exercise and skill identifiers together with binary correctness labels, but do not include the actual content of the exercises.

Several recent datasets have begun to provide richer descriptions of educational interactions. MIAAM belongs to this new generation of datasets. It contains 7.24 million mathematics learning interactions from 45,848 students across two educational platforms, together with textual exercise information, submitted answers, response durations, pedagogical metadata, feedback messages, and, when available, screenshots [2]. Such datasets make it possible to train predictive models that use substantially more information than exercise identifiers and binary outcomes. The main difficulty is then to represent these heterogeneous inputs without discarding their semantic content. Preserving this content is particularly important if the model is expected to relate exercises through their meaning and to remain robust when encountering exercises that were rare or absent during training.

Pretrained language models provide a natural basis for addressing this problem. Approaches such as Language Model-based Knowledge Tracing and Next Token Knowledge Tracing represent exercises, and sometimes complete interaction histories, as text [7, 11]. Unlike identifier-based models, they can exploit semantic relations between exercise contents. However, their ability to represent a student history remains constrained by the context available to the language model. Encoder-based approaches such as LKT are limited to short sequences, while directly serializing a long and information-rich history for a causal language model rapidly consumes its context window and increases its computational cost. This creates a trade-off between preserving the information contained in previous interactions and representing this history in a form that can be processed efficiently.

MAGELLAN provides a complementary example of semantic competence prediction. It uses textual goal descriptions to estimate the competence and learning progress of an autonomous agent [5]. However, its predictor is trained for a single learner. In a population-level educational setting, maintaining a separate language model for every student would not be practical. A shared model must instead distinguish students through information provided at inference time, naturally leading to the use of their previous interactions as context.

These constraints lead to two research questions:

- RQ1. History representation.** How can a long and semantically rich interaction history be encoded into a compact representation that preserves its relevant information and can be processed efficiently by a pretrained language model?
- RQ2. History conditioning.** How does the mechanism used to provide this representation to the

language model affect its ability to exploit the history, and which mechanism is the most effective?

Investigating these questions led us to develop *Shandess*. Its History Encoder combines semantic exercise representations computed in advance with interaction-specific signals available during training and inference. It produces compact history tokens that can be provided to the causal language model through interchangeable conditioning mechanisms, allowing history representation and history conditioning to be studied separately. The two principal variants are called *Shandess Soft Prompt* and *Shandess HiCA*. These names denote the conditioning methods; HEC configurations and training-history thresholds are specified separately when needed.

The main contributions of this work are:

- The design of a History Encoder that combines semantic exercise representations with observed student behavior to provide compact interaction histories to a pretrained language model.
- An empirical study of two history-conditioning mechanisms, complemented by experiments on architectural and training choices, including training-data distributions and LoRA versus DoRA.
- An evaluation of response prediction on MIAAM, including comparisons with KT baselines, the use of additional history, prediction for unseen students and exercises, and the inference cost of a compact population-level model.

The remainder of this report is organized as follows. Section 2 explains how KT and AP models can support the construction of personalized learning trajectories and identifies the main requirements associated with this application. Section 3 reviews existing approaches to Knowledge Tracing and semantic response prediction. Section 4 presents the MIAAM dataset and formalizes the two prediction tasks considered in this work. Sections 5 and 6 describe the progressive design of *Shandess* and its training procedure. Section 7 compares the resulting models with the selected baselines and assesses their suitability for the intended use. Section 8 presents future work, and Section 9 concludes the report.

2 Intended Use, Model Requirements, and Prediction Tasks

2.1 From response prediction to personalized exercise recommendation

Taken in isolation, a response-prediction model has limited direct utility for students. It predicts how a student is likely to respond to an exercise, but it neither selects an exercise nor interacts with the student. Its predictions can, however, serve as the main component of a personalized exercise-selection system.

Given a set of candidate exercises, such a system could use the response predictor to estimate the probability that the student will answer each exercise correctly. It could then select an exercise whose predicted success probability is close to a predefined target. This target would determine the desired level of difficulty: the selected exercise should remain accessible to the student while still providing an opportunity to learn. Repeating this process after every interaction would produce a trajectory that adapts as the student’s history evolves.

This initial selection mechanism would only use predicted success. It could then be extended with additional criteria, such as the skill being practiced, the type and format of the exercise, prerequisite relations, diversity among consecutive activities, or constraints defined by teachers. The response predictor would remain responsible for estimating the student’s response, while a separate policy would combine these predictions with the other criteria to select the next exercise.

The long-term purpose of such a system would be to facilitate personalized learning at national scale. It would occupy a role similar to older adaptive methods such as ZPDES, which is currently used

by EvidenceB in the educational platforms from which MIAAM was collected [2, 3]. ZPDES selects activities using empirical learning progress and expert-defined pedagogical structures. The proposed system would instead rely on predictions made at the exercise level from the exercise content and the student’s interaction history, while remaining compatible with additional pedagogical constraints.

This intended use determines the properties required from the response-prediction model developed in this work. A model designed only to obtain good results on a fixed benchmark would not necessarily be suitable for repeated use by a national educational platform.

2.2 Requirements for large-scale deployment

The main constraints associated with this application are the following:

- **Predictive quality and calibration.** The exercise-selection policy depends directly on the probabilities produced by the response predictor. The model must therefore predict student responses accurately, but its probabilities must also be meaningful. If it systematically overestimates or underestimates the probability of success, selecting exercises around a target success rate becomes unreliable. Calibration is consequently important in addition to classification accuracy.
- **Adaptation to new students.** A national platform continuously encounters students who were not present during training. The model must produce useful initial predictions from population-level patterns and then adapt them as the first interactions of the new student become available. This requires a single shared model that can condition its predictions on each student’s history, rather than a separately trained model for every learner.
- **Integration of new exercises.** Educational content evolves as exercises are added, modified, or generated. Integrating a new exercise should not require learning a new identifier and retraining the complete model. The response predictor should instead construct its representation from the content and metadata of the exercise. This architectural property makes integration possible, although the quality of predictions on exercises unseen during training must still be evaluated separately.
- **Use of long histories.** A student may interact with the platform over several months or years. Restricting the model to a few recent exercises can discard information about previously encountered concepts, recurring errors, or changes in competence. The model must therefore exploit long histories without representing every interaction as a large textual prompt whose cost grows rapidly with the trajectory length.
- **Computational efficiency.** Exercise recommendation requires repeated predictions, potentially for several candidate exercises and for a large number of students. Dependence on a frontier model or an external proprietary service would make such use expensive and difficult to deploy. The model must remain sufficiently small for low-cost inference and, ideally, be usable locally on accessible hardware. Exercise information that does not depend on the student should also be computed in advance whenever possible.

These constraints define the response-prediction model sought in this work and provide criteria for evaluating its practical relevance beyond performance on MIAAM. They guided the development of the final architecture, Shandess, which is introduced progressively in Section 5.

2.3 Why study both KT and AP?

The response-prediction architecture developed in this work is trained and evaluated on two tasks: Knowledge Tracing (KT) and Answer Prediction (AP). Since both tasks concern the same student interactions, their simultaneous use requires justification.

In KT, the model predicts whether the student will answer the current exercise correctly. The output space is binary and does not distinguish between different incorrect responses. This makes KT comparatively simple to train and evaluate. It is also the standard formulation used by most existing student-modeling methods, which allows the proposed architecture to be compared with established baselines.

In AP, the model instead predicts the exact answer selected by the student among the available options. This is a more detailed prediction problem, especially when an exercise contains many possible answers. Its main interest is that two incorrect answers can carry different information. They may correspond to different strategies, calculation errors, or misconceptions that are indistinguishable under the binary KT label. AP therefore provides a more suitable basis for studying and eventually interpreting the errors made by a student.

Evaluating both tasks also serves an architectural purpose. The History Encoder is intended to represent the exercise and interaction history independently of the final prediction task. Training the same general architecture on both KT and AP tests whether it remains effective under different output spaces. A future transfer experiment from KT to AP, or from AP to KT, with a fixed History Encoder could further test whether the learned representation is task-independent. Such transfer is not evaluated in this report.

Despite their differences, KT and AP remain closely related. Let a_t be the answer selected by the student and a_t^* the correct answer. The KT target is obtained directly from the AP target:

$$y_t = \mathbb{I}[a_t = a_t^*].$$

An AP model therefore induces a KT prediction by assigning a probability to the correct answer. Conversely, a multiple-choice prediction can be formulated as a collection of binary decisions, although a single KT prediction cannot recover which incorrect answer was selected. KT can thus be viewed as a coarser version of AP. The two tasks address different experimental and practical needs while remaining two formulations of the same underlying problem: predicting how a student will respond to an exercise.

3 Background and Related Work

The purpose of this section is not to provide an exhaustive review of knowledge tracing. Instead, we focus on the models that constitute our main baselines or are closely related to the objective defined in the previous section. Beyond predictive performance, we examine whether each approach can exploit the semantic content of exercises, generalize to unseen exercises, adapt to new students from a limited history, and remain sufficiently efficient for large-scale deployment.

3.1 Classical and structured knowledge tracing

Early knowledge tracing methods generally represent student knowledge through a predefined set of knowledge components. Bayesian Knowledge Tracing (BKT), for instance, associates each knowledge component with a latent binary mastery state and models how this state evolves as the student interacts with exercises [4]. This provides a simple and interpretable representation of student knowledge, but requires exercises to be assigned to the appropriate knowledge components beforehand. It also assumes that exercises associated with the same component provide equivalent evidence about the student’s mastery.

Deep Knowledge Tracing (DKT) replaces this explicit probabilistic model with a recurrent neural network [13]. At each time step, the model receives the identifier of the attempted knowledge component together with the correctness of the student’s response. Its hidden state provides a compact summary of the previous interactions, from which the probability of success on the next knowledge component is

predicted. Since the same model is trained across a population, the hidden state can be initialized for a new student and progressively adapted as new interactions become available.

DKT is compact and inexpensive at inference time because the history is summarized recurrently instead of being processed again for each prediction. However, its representation of exercises is rigid. The model operates on a fixed vocabulary of knowledge components and does not have direct access to the content of the exercises. Consequently, two exercises associated with the same component are indistinguishable to the model, even if they differ in wording, format, difficulty, or required reasoning. A genuinely new component also changes the input or output vocabulary and therefore generally requires modifying and retraining the model.

qDKT addresses part of this limitation by moving from skill-level to question-level prediction [14]. Instead of estimating one success probability for each knowledge component, it estimates a separate probability for each exercise. This makes it possible to represent differences between exercises associated with the same skill. Because question-level observations are sparser, qDKT introduces a graph-Laplacian regularizer that encourages similar questions to receive similar predictions. The similarity graph may be constructed from skill annotations and can, in principle, also incorporate textual similarity. The model further uses a fastText-inspired initialization based on sequences of question identifiers and outcomes.

Despite this finer granularity, the original qDKT model remains based on a fixed set of exercise identifiers. Its fastText-inspired initialization is learned from interaction sequences rather than from the natural-language content of the exercises. Moreover, adding a new exercise requires introducing a new input representation and a new component in the question-level output space. qDKT is therefore a strong and efficient question-level baseline, but it does not directly solve the problem of representing unseen exercises from their content.

Graph-based Knowledge Tracing (GKT) introduces a different form of prior information by explicitly representing relations between knowledge components [10]. Knowledge components are represented as nodes of a graph, and graph neural network operations propagate information between the component involved in the current interaction and its neighbors. This allows the update of the student state to depend on known relations between concepts instead of treating all components independently.

In MIAAM, GKT is provided with the activity dependency graph released with the dataset [2]. This graph contains meaningful pedagogical information: it describes prerequisite relations and the position of activities within the curriculum. However, this information is structural rather than semantic. GKT does not directly process the text, interface, or expected reasoning of an individual exercise. An exercise can be handled if it is mapped to an existing activity, but the model cannot distinguish it from other exercises belonging to the same activity. Adding a genuinely new activity also requires placing it in the graph and adapting the corresponding model parameters.

DKT, qDKT, and GKT therefore provide complementary structured baselines. DKT efficiently summarizes interaction histories at the activity or skill level, qDKT distinguishes individual exercises, and GKT incorporates relations between activities. Nevertheless, all three remain dependent on a predefined vocabulary or structure. In particular, they cannot directly represent an exercise that was not observed during training from its content alone. Such an exercise must generally be mapped to an existing activity or added to the model’s vocabulary, potentially requiring manual annotation and retraining. This constitutes the exercise cold-start problem.

3.2 Knowledge tracing from exercise content

A natural way to address exercise cold start is to construct exercise representations from their content. Two exercises with related meanings can then receive related representations even if they have different identifiers, while an unseen exercise can be represented without introducing a new learned embedding. Content-based representations may therefore reduce the need for manual annotation and allow models

to generalize across exercises from their semantic similarities.

The content-based qDKT baseline evaluated on MIAAM constitutes a first step in this direction. In addition to the exercise-identifier version, MIAAM evaluates a qDKT variant in which textual exercise descriptions are encoded with a pretrained Sentence Transformer [2]. This replaces a purely symbolic exercise representation with a vector containing some semantic information. It also avoids relying entirely on manually assigned activity labels.

However, the text is encoded independently of the student history and is reduced to a single fixed embedding before being processed by the knowledge tracing model. The semantic information accessible to qDKT is therefore limited to what is preserved by this embedding. Moreover, replacing the input representation does not by itself remove every dependency on the task-specific question vocabulary. The MIAAM experiments do not show a clear improvement over the exercise-identifier variant, suggesting that providing a semantic embedding is not sufficient if the rest of the architecture is not designed to exploit it effectively.

Language model-based Knowledge Tracing (LKT) goes further by representing both exercises and interaction histories directly as text [7]. Each interaction is serialized using the textual description of the knowledge component, the question, and a special token indicating whether the answer was correct or incorrect. Encoder-based pretrained language models such as BERT, RoBERTa, or DeBERTa are then fine-tuned to predict masked correctness tokens.

This formulation has two important advantages. First, the language model can exploit semantic relations between exercises instead of learning every exercise representation from scratch. Second, its pretrained representation provides useful information when little task-specific data or only a short student history is available. LKT reports improved robustness in low-data and short-history settings, as well as cross-dataset zero-shot capabilities that are unavailable to identifier-based DKT models.

The main constraint of LKT is the context length of its encoder. The experiments use sequences of at most 512 tokens. Because the text of every previous exercise must be included in the same sequence, a semantically rich exercise may consume a substantial fraction of this budget. The current exercise and the student history also compete for the same context window. Consequently, the number of usable interactions may be much smaller than the number of interactions available in the original trajectory, especially for exercises with long descriptions.

Next Token Knowledge Tracing (NTKT) follows the same general motivation but replaces the bidirectional encoder with a causal language model [11]. The student’s previous interactions and the current exercise are serialized as a tagged textual sequence, and correctness prediction is formulated as next-token prediction. During training, the loss is applied only to the tokens representing the correct or incorrect outcome, while the remaining tokens provide context to the model.

NTKT uses decoder-only language models and supports substantially longer sequences than LKT. The reported experiments use context lengths of up to 15,000 tokens, allowing the model to process longer textual histories. The work also explicitly evaluates two cold-start settings. In the student cold-start setting, the model must adapt its predictions to previously unseen students as their histories grow. In the exercise cold-start setting, entire exercises are excluded from training and must be represented from their text at evaluation time. These experiments directly study the capacity of pretrained linguistic representations to generalize across both students and exercises.

This flexibility comes with a higher computational cost. Unlike recurrent KT models, which maintain a compact hidden state, LKT and NTKT serialize the interaction history inside the language-model context. The history must therefore be tokenized and processed when making a prediction, and the cost increases with its length. NTKT reduces the cost of training through parameter-efficient adaptation, but still relies on language models containing one to several billion parameters. It also focuses on binary correctness prediction and does not evaluate the prediction of the student’s exact answer.

LKT and NTKT demonstrate that pretrained language models can provide exercise representations

that are more transferable than identifiers. They also show that semantic content is particularly useful in cold-start settings. However, both approaches represent the history as an explicit textual sequence processed by the same model as the current exercise. Their capacity to exploit richer histories is consequently constrained either by a short context window, as in LKT, or by the computational cost of processing a long context, as in NTKT.

3.3 Language models for competence and learning-progress prediction

Instead of training a dedicated knowledge tracing architecture, another possibility is to directly prompt a general-purpose foundation model with the student’s history and the current exercise. The MIAAM benchmark evaluates this approach using frontier language and vision-language models with different levels of access to exercise content and pedagogical information [2]. These models receive textual descriptions or screenshots of exercises, and may additionally receive expert-authored pedagogical descriptions.

Prompted foundation models have direct access to much richer information than classical KT models. They can process the content of a previously unseen exercise without adding a new identifier, and the same generative interface can be used for both binary knowledge tracing and answer prediction. This makes them an important reference for evaluating whether pretrained semantic capabilities are sufficient to model individual students.

In practice, however, the size of these models strongly restricts their use. In the MIAAM experiments, computational and financial constraints limit textual histories to 100 interactions and visual histories to 10 interactions. Evaluation is also performed on a subset of the test interactions. Despite receiving richer inputs and being orders of magnitude larger than classical KT models, these models do not consistently outperform the best structured baselines. Their inference cost is particularly problematic in the intended application, where a prediction may be required for many candidate exercises and for a very large number of students.

MAGELLAN studies a related problem from the perspective of automatic curriculum learning [5]. It is not a knowledge tracing model for human students. Instead, it estimates the competence and learning progress of an autonomous language-model agent on goals expressed in natural language. A competence estimator is trained online from recent goal–outcome pairs. By exploiting the semantic representation of the goals, an update performed after attempting one goal can also affect competence estimates for semantically related goals.

MAGELLAN compares current and previous versions of this estimator to approximate absolute learning progress. These progress estimates are then used by a goal-selection policy to prioritize goals on which the agent is currently improving. It therefore illustrates how semantic competence estimates can be transformed into a learning trajectory, rather than merely used to predict the next outcome.

This connection is particularly relevant to the intended use described in Section 2. Nevertheless, MAGELLAN operates in a different setting. Its estimator follows the evolution of one autonomous policy and is continuously updated from that policy’s own experience. It does not learn a single population-level model that can be conditioned on the explicit interaction history of an arbitrary student. The history is used as an online training buffer rather than encoded into a representation supplied at inference time. Applying the same approach independently to a national population of students would also require maintaining and updating separate model parameters for individual learners.

Prompted foundation models and MAGELLAN thus address two complementary parts of the problem. Prompted models can condition predictions on a semantically rich history, but their inference cost prevents large-scale use. MAGELLAN connects semantic competence prediction to trajectory construction, but its learner-specific online adaptation does not directly transfer to population-level student modeling.

3.4 Comparison with the intended-use requirements

Table 1 summarizes the main properties of the approaches discussed above. The comparison is qualitative: the reported costs correspond to the relative architectural requirements of the models rather than to measurements performed on identical hardware. In particular, the ability to represent a new exercise should not be confused with demonstrated zero-shot prediction. A model may accept a content embedding for a new exercise while still relying on an output space or training procedure tied to the exercises observed during training.

Model	Exercise representation	Unseen exercise	History	Target	Cost
DKT	Knowledge-component or activity identifiers	Only through an existing component	Recurrent hidden state	KT	Low
qDKT	Exercise identifiers	Not supported by the fixed output space	Recurrent hidden state	KT	Low
GKT	Activity or concept graph	Requires mapping or insertion into the graph	Recurrent graph state	KT	Low–moderate
qDKT-text	Trainable embeddings initialized from text (local)	Semantic input possible, but zero-shot prediction is not demonstrated	Recurrent hidden state	KT	Low
LKT	Knowledge-component and exercise text	Possible from text; item-level cold start is not isolated	At most 512 tokens in the reported experiments	KT	Moderate
NTKT	Exercise text and metadata	Explicitly evaluated	Up to 15,000 tokens in the reported experiments	KT	High
Prompted foundation models	Raw text, images, and pedagogical metadata	Supported in principle	Restricted in practice by inference cost	KT/AP	Very high
MAGEL-LAN	Natural-language goals	Explicitly evaluated for new goals	Online buffer and previous model states	Competence / LP	Per-agent adaptation

Table 1: Qualitative comparison of related approaches under the requirements introduced in Section 2.

The reviewed approaches reveal a consistent trade-off. Identifier- and graph-based KT models are compact and can summarize long sequences efficiently, but they cannot directly preserve the semantic content of individual exercises. Content-aware KT models improve generalization by using pretrained linguistic representations, but either compress the exercise semantics into a fixed vector or process the complete history as an expensive textual context. Prompted frontier models provide the most flexible interface, including support for answer prediction, but their cost is incompatible with repeated inference at national scale. Finally, MAGELLAN demonstrates how semantic competence estimates can guide trajectory construction, but relies on learner-specific online adaptation rather than a population-level history-conditioned model.

None of these approaches simultaneously provides a semantic representation of individual exercises, efficient access to a long and information-rich interaction history, support for both knowledge tracing and answer prediction, and a sufficiently small model for local deployment. This remaining gap motivates the architecture and experiments introduced in the following sections.

4 MIAAM

4.1 MIAAM dataset

MIAAM is a dataset of mathematics learning interactions collected from two adaptive educational platforms deployed in French schools: Adaptiv’Math (AM), which targets early primary education, and MIA Seconde (MIA), which targets Grade 10 remediation. The complete release contains approximately 7.24 million attempts from 45,848 students and combines interaction logs with exercise content, pedagogical metadata, screenshots, and prerequisite graphs [2].

The models evaluated in this report do not operate directly on the complete MIAAM release. They use a preprocessed version organized into four files: two interaction tables, an exercise table, and a file containing textual exercise descriptions.

Interaction tables. The interactions are divided into `interactions_train.parquet` and `interactions_test.parquet`. Each row represents one attempt made by a student on an exercise. It contains the anonymized identifiers of the student, classroom, exercise, session, and corresponding module or playlist; the timestamp and index of the attempt; the submitted answer; whether the answer was correct; its duration; the work mode; and the platform from which the interaction originated.

The tables are enriched during preprocessing with the module and activity associated with each exercise. Integer versions of the student, exercise, and activity identifiers are also added for models that require finite vocabularies of discrete identifiers. The temporal fields and attempt indices make it possible to reconstruct each student’s interaction history in chronological order.

The split is performed at the student level. Consequently, no student appears in both tables. This setting evaluates whether a model trained on one population can use the history of a previously unseen student, rather than continuing trajectories already observed during training.

Exercise table. The file `maths_exercises_table.parquet` associates each exercise with its game-play type and its position in the pedagogical hierarchy: module, objective, and activity. Names and expert-written pedagogical intents are available at both the objective and activity levels.

The `content` field contains a serialized representation of the exercise, including its instruction, question, possible answers when applicable, correct answer, feedback, and references to visual resources. Exercises are joined to interactions using the pair `(source, exercise_id)`, since identifiers should not be assumed to be shared across platforms.

The table contains 7,151 rows corresponding to 7,130 distinct source–exercise pairs. Twenty-one pairs occur in more than one row, but none is associated with conflicting gameplay types. Among these exercises, 6,936 occur in at least one of the two interaction splits.

Textual descriptions. The file `descriptions.json` is a dictionary mapping 7,118 exercise identifiers to free-form textual descriptions. Shandess uses these descriptions directly; the original screenshot files are not loaded by the model. While twelve exercises from the exercise table do not have a description, none of them occurs in the interaction tables used in this report. Therefore, all 6,936 exercises encountered during training or evaluation have an associated description.

Relation to the prediction tasks. Both student-modeling tasks are constructed from the same chronological interaction records. For Knowledge Tracing (KT), the target associated with the current exercise is the binary `data_correct` field. For Answer Prediction (AP), the target is the option selected by the student in `data_answer`. Previous rows from the same student form the interaction history, while the exercise table and textual descriptions provide the semantic information associated with the current and previous exercises.

4.2 Dataset distributions relevant to student modeling

The following statistics describe the preprocessed interaction tables actually used in the experiments, rather than the complete MIAAM release. Together, the train and test splits contain 6,266,035 attempts from 38,520 students and 6,936 distinct source–exercise pairs. No correctness, answer, or duration value is missing.

Split	Attempts	Students	Exercises	Success rate
Train	5,044,292	30,816	6,881	78.69%
Test	1,221,743	7,704	6,269	78.60%
Combined	6,266,035	38,520	6,936	78.67%

Table 2: Main statistics of the preprocessed interaction splits. Exercises are counted as distinct (`source`, `exercise_id`) pairs.

The student split is exactly 80/20, with 30,816 students in the training set and 7,704 in the test set. Although the students are disjoint, most exercises are shared: 6,214 of the 6,269 test exercises also occur in the training set. The remaining 55 unseen exercises, representing 0.88% of the test exercise set, all come from AM.

The exercise cold-start evaluation in Section 7.5 retains admissible first encounters from 54 of these 55 identities. The dataset-level count of unseen exercises therefore differs from the number represented among evaluated targets.

The two sources are not equally represented. AM accounts for 67.95% of the interactions and 77.98% of the exercises, while MIA contributes 32.05% of the interactions and 22.02% of the exercises. Their success rates also differ substantially: 81.28% for AM and 73.13% for MIA.

Source	Attempts	Share	Students	Share	Exercises	Success
AM	4,257,868	67.95%	20,826	54.07%	5,409	81.28%
MIA	2,008,167	32.05%	17,694	45.93%	1,527	73.13%

Table 3: Distribution of interactions, students, and exercises by source.

Overall, 4,929,433 interactions are correct and 1,336,602 are incorrect. However, this global success rate hides substantial differences between exercise formats. The dataset contains sixteen gameplay types. Multiple-choice exercises are the most frequent individual category, with 2,054,990 attempts, followed by `GRADUATED_RULER`, `DROP_COMPARE`, and `DROP_QUANTITY`. Together, these four categories account for approximately 87.6% of all interactions.

Multiple-choice exercises represent 32.80% of the attempts but only 20.93% of the exercises. They are also considerably easier according to the observed success rates: 91.47% of MCQ attempts are correct, compared with 72.42% for the remaining gameplays. This difference is particularly large for AM, whose two-option MCQs have a success rate above 95%.

Source	Gameplay group	Attempts	Within-source share	Success rate
AM	MCQ	1,497,041	35.16%	95.18%
AM	Other	2,760,827	64.84%	73.74%
MIA	MCQ	557,949	27.78%	81.53%
MIA	Other	1,450,218	72.22%	69.91%
All	MCQ	2,054,990	32.80%	91.47%
All	Other	4,211,045	67.20%	72.42%

Table 4: Distribution and success rate of multiple-choice and other gameplay types.

The answer field follows the expected single-option representation for 2,053,499 MCQ attempts, corresponding to 99.93% of them. The remaining 1,491 attempts contain either an invalid answer representation or multiple selected options.

The number of options reported in Table 5 is inferred from observed answer indices. More precisely, it is defined as one plus the highest option index selected for the corresponding exercise. This quantity can underestimate the declared number of options if an option was never selected. However, every classifiable AM exercise has two observed options, whereas MIA contains exercises with two, three, and four observed options.

Source	Observed options	Exercises	Valid attempts	Success rate
AM	2	884	1,496,750	95.20%
MIA	2	267	264,627	90.31%
MIA	3	98	102,532	72.14%
MIA	4	201	189,590	74.86%

Table 5: Distribution of the empirically observed number of MCQ options. Only attempts containing one valid answer index are included.

Two of the 568 MIA MCQ exercises have no valid single-option answer in the interaction tables and therefore cannot be assigned an empirical number of options using this procedure.

Finally, the interaction histories are long and highly variable. A student has 162.7 interactions on average, but the median is only 84. The 90th percentile reaches 394 interactions, the 95th percentile reaches 600, and the longest trajectory contains 4,876 interactions. Histories are longer on AM than on MIA: their respective medians are 92 and 78 interactions, and their 95th percentiles are 785 and 345. These distributions confirm that the dataset contains histories that are much longer than those that can be represented by simply inserting every interaction verbatim into a language-model prompt.

4.3 Prediction metrics

We distinguish the student’s observed response from the model’s prediction. Let y_t denote the prediction target, $\hat{y}_t$ its predicted value, and c_t the correctness of the student’s response. In KT, $y_t = c_t$ is binary; in AP, y_t is the selected option index. We report

$$\text{Acc}_0 = \mathbb{P}(\hat{y}_t = y_t \mid c_t = 0), \quad (1)$$

$$\text{Acc}_1 = \mathbb{P}(\hat{y}_t = y_t \mid c_t = 1), \quad (2)$$

$$\text{MeanAcc} = \frac{1}{2}(\text{Acc}_0 + \text{Acc}_1). \quad (3)$$

These quantities are estimated by the corresponding empirical proportions on the evaluation sample. MeanAcc gives equal weight to incorrect and correct student responses. It equals global accuracy when these two groups have equal size. For KT, this is balanced accuracy; for AP, Acc_0 measures whether the model predicts the exact wrong option selected by the student. Unless a table specifies a different grouping, bold values mark the best observed score in each metric column, including ties. They do not by themselves establish a statistically resolved difference.

5 Designing Shandess

The requirements identified in Section 2 do not only concern predictive performance. The model must also adapt to new students, represent new exercises from their content, use histories that may contain hundreds of interactions, and remain small enough for repeated local inference. Shandess was developed progressively around these constraints. We first tested the most direct approach, in which a language model receives the complete interaction history as text. We then separated the history from the text prompt, designed a dedicated History Encoder (HEc), and compared two ways of injecting its output into the language model. Finally, we investigated ways to improve the history representations produced by HEc.

Configuration names. We call the two conditioning methods **Shandess Soft Prompt** and **Shandess HiCA**. Within each method, we distinguish two axes of variation: the HEc architecture (tokens per interaction, metadata and an optional history Transformer), and the minimum history admitted during training. Architectural differences are stated explicitly rather than assigned additional model names. The suffix h1 denotes a minimum of one preceding interaction during training, and h10 a minimum of ten; neither suffix fixes the history length. Section 6.4 explains why both thresholds are studied. Training source mixtures and initialization paths are reported separately as protocol details.

5.1 Using a language model as the prediction model

Our starting point is to formulate KT and AP as language-model prediction tasks. This choice is motivated by three properties of pretrained language models. First, their prior knowledge of language and elementary mathematics may provide useful heuristics before the student-specific signal has been learned. Second, instruction tuning makes the task easy to specify through a prompt and may reduce the amount of task-specific training required. Finally, the model can process the content of a new exercise directly, rather than requiring a learned exercise identifier.

We use Qwen2.5-0.5B-Instruct as the prediction model. This model is small enough to satisfy the deployment constraint more closely than a frontier language model, while still providing a causal language-model interface. Preliminary versions of the pipeline used a bidirectional encoder. We moved to a causal model because it provides a longer usable context at a comparable model size and naturally supports the same generative interface for both tasks. This change increases the amount of history that can be represented, but does not remove the context-length problem discussed below.

The language model is adapted with LoRA [6]. LoRA was selected as a simple and widely used parameter-efficient fine-tuning method. At this stage, the objective was to test the use of a language model for student prediction, not to optimize the adaptation method itself. The comparison with DoRA is therefore postponed until Section 5.4.4.

For each prediction, the prompt contains a system instruction, a chronological description of the previous attempts, and the current exercise. Each past attempt includes the textual exercise description, its gameplay type, the correctness of the response, the response duration, and, for multiple-choice exercises, the selected option when it is available. The current exercise is represented by its textual description. For AP, the prompt also specifies the number of available options. The complete prompt templates and an example are provided in Appendix F.

For KT, the model is instructed to output a single token: 0 for an incorrect response and 1 for a correct response. For AP, it outputs the index of the option that the student is predicted to select, with indices starting from zero. The objective is therefore to predict the student’s behavior, rather than to solve the exercise and return its correct answer. During training, the loss is applied only to the expected output tokens. KT evaluation extracts the logits for the two response tokens. AP evaluates the predicted option index, with an invalid index counted as an incorrect prediction.

We refer to this direct formulation as NORMAL. Conceptually, it asks a language model to read the student’s history and predict their next response:

$$p(y_t \mid e_t, H_t) = p_{\text{LM}}(y_t \mid \text{Prompt}(e_t, H_t)), \quad (4)$$

where e_t is the current exercise and $H_t = \{(e_i, r_i)\}_{i < t}$ is the previous interaction history.

This simple baseline already reaches a mean validation accuracy of 0.7694 on the recent balanced KT validation set. The result is sufficiently high to show that a small language model can learn the task, and the model benefits from an interface that supports both KT and AP without changing its backbone. However, NORMAL also makes the main practical limitation measurable: the history is expensive because each interaction is represented by many text tokens.

In our experiments, NORMAL uses a context budget of 2,048 tokens and windows containing at most 40 past interactions. Even within this restricted window, the complete prompt does not always fit. The most recent interactions are then retained until the token budget is exhausted. Increasing the number of interactions would either discard the oldest part of the trajectory or require a larger context. Using a substantially larger model only to obtain a larger context would directly conflict with the inference constraint defined in Section 2.

The same representation also increases computation. Every detail of every past exercise must be tokenized and processed again for each new target, even though most exercise contents are identical across many prediction windows. The attention cost also grows with the total number of prompt tokens. In the configuration used here, training NORMAL required three successive runs of 2 h 00 min 07 s, 2 h 00 min 08 s, and 2 h 00 min 13 s on four H100 GPUs. This corresponds to a cumulative wall-clock time of 6 h 00 min 28 s and a direct compute cost of 24 h 01 min 52 s H100. Evaluation required 46.663 ms per model call (median, batch one, stored $N = 200$, input preparation excluded). Training costs for the HEc-based modes, including the prerequisite stages reused by some pipelines, are reported in Section 6.

The direct textual formulation is therefore a useful conceptual baseline, but not a satisfactory final architecture. We still want the language model to read the current exercise, because this is where its semantic and mathematical prior is most useful. The history, in contrast, must be provided through a more compact representation.

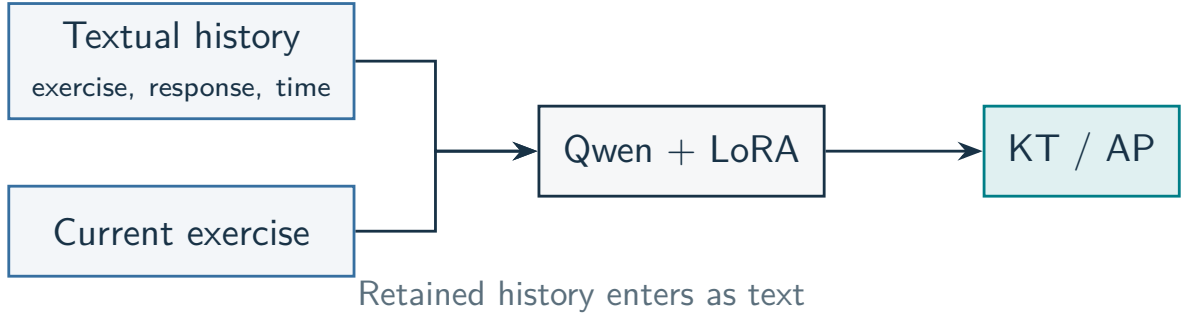


Figure 1: Normal supplies the retained interaction history and the current exercise as text to Qwen.

5.2 Separating the history from the text prompt

The central idea of Shandess is to separate the current exercise from the student history. The current exercise remains in the text prompt and is processed directly by the language model. The previous interactions are removed from this prompt and compressed into a sequence of vectors by a dedicated History Encoder, denoted HEC. The prediction becomes

$$p(y_t | e_t, H_t) = p_{\text{LM}}(y_t | \text{Prompt}(e_t), \text{HEC}(H_t)). \quad (5)$$

This separation relies on the hypothesis that the complete textual form of every past exercise is not required at every prediction step. The model needs to know what the exercise was about and how the student interacted with it, but it does not necessarily need to reconstruct every word, interface element, or numerical value appearing in the original statement. Similarly, the exact duration or position of an attempt can be represented as a numerical feature rather than verbalized in the prompt. Compression necessarily removes some information, but it may preserve the information that is useful for estimating the student’s state while substantially reducing the input length.

Separating exercise information from student-specific information also makes it possible to avoid repeated computation. The description of an exercise is independent of the student and can be encoded once. We use the frozen Qwen3-Embedding-0.6B sentence encoder to map every description in `descriptions.json` to a 1,024-dimensional vector. These vectors are precomputed and stored. At training or inference time, HEC only retrieves the vector of each exercise and fuses it with the behavior observed for the current student. A newly added exercise requires one new semantic embedding, but not a new learned identifier or a complete retraining of the model.

Initial History Encoder. Figure 2 summarizes the first version of HEC. For each past interaction i , the encoder receives a semantic exercise vector s_i and six behavior features:

- whether the response was correct;
- the selected answer index, divided by 10 when it is available;
- an indicator specifying whether an answer index is available;
- an indicator specifying whether the exercise is multiple-choice;
- $\log(1 + d_i)/8$, where d_i is the response duration in seconds;
- the relative position of the interaction in the selected history.

The behavior vector is first mapped to a 64-dimensional representation by a two-layer MLP. It is concatenated with the 1,024-dimensional exercise vector, then passed through a second MLP to produce a 256-dimensional interaction token. A learned positional embedding is added to distinguish the order of the attempts. Padding tokens are masked throughout the model.

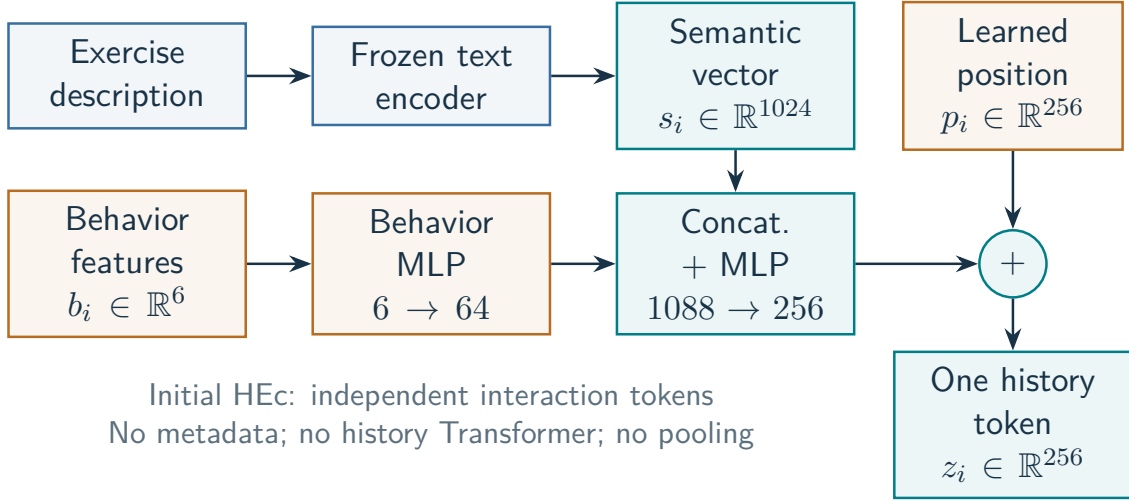


Figure 2: Initial History Encoder. Exercise descriptions are encoded once by a frozen sentence encoder. For each student, the retrieved exercise vector is fused with the features of the corresponding attempt to produce one token. This first version does not contain a Transformer: interaction tokens are computed independently before being passed to the language model.

This initial version deliberately uses one token per interaction and no Transformer inside HEc. It therefore isolates the effect of replacing the textual history with compact vectors before introducing more complex sequence-level processing. With a history of 200 attempts, the language model receives at most 200 history tokens instead of the potentially thousands of tokens required to serialize the same exercises and responses as text.

HEc solves the representation problem, but does not determine how its output should condition the language model. The next step is therefore to compare different injection mechanisms while keeping the role of HEc unchanged.

5.3 Injecting history tokens into the language model

We consider two mechanisms for providing the HEc tokens to Qwen. Shandess Soft Prompt inserts them once at the input of the language model. Shandess HiCA instead makes them available repeatedly through cross-attention layers. Before comparing these two mechanisms, we introduce a control that removes the history completely.

5.3.1 Passthrough

The PASSTHROUGH mode uses the same current-exercise prompt and the same LoRA-adapted Qwen model as the HEc-based modes, but ignores the complete student history. It is not intended as a competitive student model. Its role is to separate two possible sources of predictive performance.

First, the language model can exploit exercise-level and population-level regularities without observing the student. It may recognize an easy exercise, estimate which option is generally attractive, or learn different base rates for AM and MIA. We refer to these regularities as exercise and dataset heuristics. Second, a history-conditioned model may additionally exploit student-specific information.

Comparing Shandess Soft Prompt and Shandess HiCA with PASSTHROUGH measures whether HEc transmits a useful signal beyond the heuristics available from the current exercise alone.

This control is particularly important on MIAAM because the observed success rate depends strongly on the source and gameplay type, as shown in Section 4.2. A high aggregate score is therefore not, by itself, evidence that the model uses the student history.

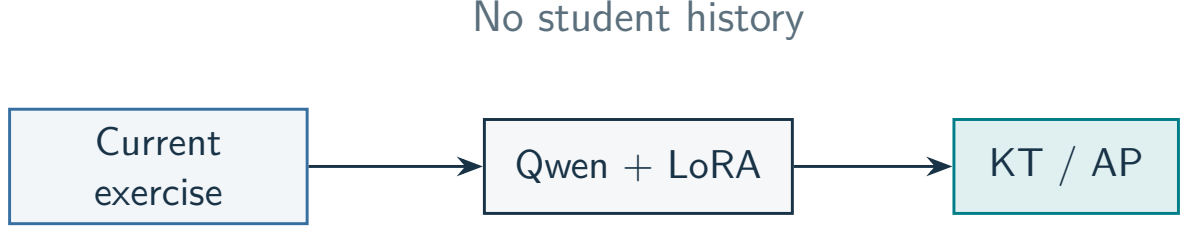


Figure 3: Passthrough adapts Qwen to predict a student response from the current exercise, without historical conditioning.

5.3.2 Shandess Soft Prompt

The most direct way to reuse the compact history is to place it in front of the current-exercise prompt. Converting the numerical vectors to text would remove most of the benefit: the tokenizer would represent their coordinates with many tokens, and Qwen would then have to recover the vector structure from a textual serialization. Instead, Shandess Soft Prompt bypasses the tokenizer.

Let $Z = \text{HEc}(H_t)$ denote the history tokens. A layer normalization and a learned linear projection map them from the HEc dimension to the 896-dimensional embedding space of Qwen. A sigmoid gate controls their initial contribution. The projected history is concatenated directly with the embeddings of the tokenized current-exercise prompt:

$$E_{\text{soft}} = [\sigma(g) W_Z \text{LN}(Z); \text{Emb}_{\text{Qwen}}(\text{Prompt}(e_t))] . \quad (6)$$

The history and prompt serve as input: the model is not trained to reproduce them. The loss evaluates only the predicted answer tokens, while gradients can still flow through the history representations to train HEc.

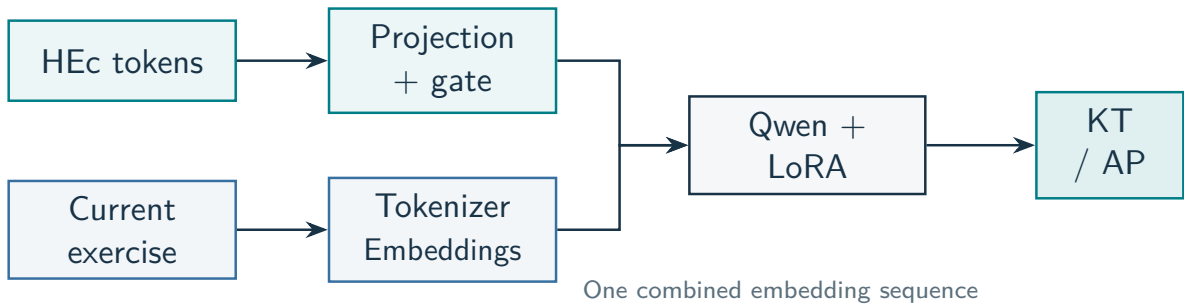


Figure 4: Shandess Soft Prompt injection. The textual instruction and current exercise are tokenized normally. The structured history is compressed by HEc, projected to Qwen’s embedding dimension, and concatenated directly with the text embeddings. A history of N interactions therefore occupies N tokens in the initial version.

Shandess Soft Prompt is simple and adds only the HEc-to-Qwen projection to the language model. It also preserves Qwen’s internal architecture. However, its history tokens are introduced only at the input. Their information must remain accessible through all subsequent causal self-attention layers. In addition, although the representation is much shorter than the textual history, the HEc tokens still

occupy positions in the context and participate in Qwen’s self-attention. These observations motivate a second injection mechanism.

5.3.3 History Cross-Attention

Shandess HiCA uses History Cross-Attention to keep the history outside the text sequence and makes it available at several depths of the language model. The design is inspired by gated cross-attention architectures such as Flamingo [1], but the external memory here represents a student’s interaction history rather than visual inputs.

The HEc tokens are first projected to Qwen’s hidden dimension. A gated cross-attention block is then inserted after every four decoder layers. Since Qwen2.5-0.5B contains 24 decoder layers, Shandess HiCA uses six such blocks. At block l , the current Qwen states form the queries, while the projected history tokens form the keys and values:

$$A_l = \text{MHA}(\text{LN}(X_l), \text{LN}(Z), \text{LN}(Z)), \quad (7)$$

$$\tilde{X}_l = X_l + \tanh(\alpha_l) A_l. \quad (8)$$

Each block also contains a feed-forward sublayer with its own residual gate:

$$X_{l+1} = \tilde{X}_l + \tanh(\beta_l) \text{FFN}(\text{LN}(\tilde{X}_l)). \quad (9)$$

The gates control how quickly the newly initialized blocks perturb the pretrained language model and make it possible to train the cross-attention before jointly adapting the remaining components.

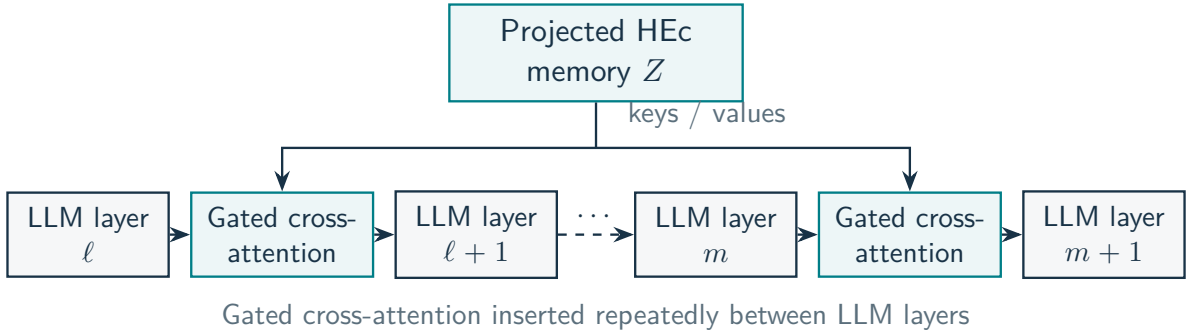


Figure 5: Shandess HiCA conditioning. The current exercise is processed by the original causal decoder. After every four decoder layers, its hidden states attend to the same external history memory through a gated cross-attention block.

Shandess HiCA gives every textual position repeated access to the history and does not consume positions in the causal text sequence. Its main disadvantage is model size. For Qwen2.5-0.5B, the six cross-attention blocks, including their feed-forward sublayers, add approximately 57.9 million parameters. For comparison, the rank-16 LoRA adapters add approximately 8.8 million parameters, the initial HEc contains approximately 0.42 million parameters, and its projection to Qwen adds 0.23 million. Shandess HiCA remains much smaller than replacing the backbone with a frontier model, but it is substantially heavier than Shandess Soft Prompt and modifies the internal execution of Qwen.

5.3.4 Intermediate results

The first experiments compare textual history, no history, and the two conditioning mechanisms with the initial one-token HEc. Table 6 summarizes this stage of the design, before changing the capacity or inputs of the History Encoder.

Model	History representation	Validation MeanAcc (%)
Normal ²⁸	Textual history	76.94
Passthrough ²⁹	No history	77.12
Shandess Soft Prompt ¹⁸	One HEc token per interaction	78.02
Shandess HiCA ¹⁹	One HEc token per interaction	78.80

Table 6: Initial KT validation results. Validations contain 10,000 targets balanced by correctness. Shandess Soft Prompt and Shandess HiCA share the same validation windows; Normal and Passthrough use their historical window configurations. Normal retains at most 40 stored interactions, versus 200 for HEc.

The textual-history baseline performs close to the no-history baseline. Under this context budget, simply serializing previous exercises is therefore less effective than supplying compact interaction tokens. The HEc-based models provide a useful way to expose the student’s previous behavior to Qwen without spending most of its text context on repeated exercise descriptions.

With the same initial HEc representation and validation windows, Shandess HiCA exceeds Shandess Soft Prompt by 0.78 percentage point. This supports studying the injection mechanism separately from the representation. One hypothesis is that intermediate exercise representations let successive cross-attention blocks retrieve different evidence from the history, making student-specific information easier to use at several levels of processing. The observed score difference does not directly test this mechanism. The result motivates retaining both mechanisms while investigating the capacity of HEc.

5.4 Improving the History Encoder

Having established that compressed history carries useful information, we investigate the representation itself: its capacity, the inclusion of pedagogical metadata, and contextualization across previous interactions.

5.4.1 Increasing the number of output tokens

Representing each interaction with a single token creates a compact interface, but may restrict the information supplied to the language model. We therefore compare the initial one-token representation with a two-token representation for each past interaction. With N observed interactions, these configurations provide N and $2N$ history tokens, respectively.

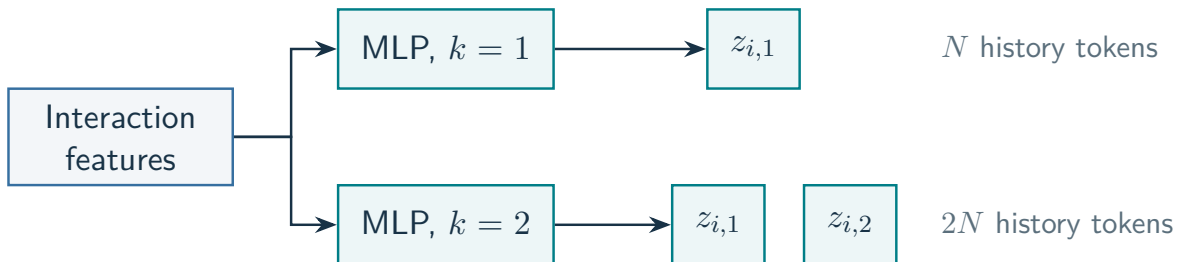


Figure 6: Increasing representation capacity per interaction and the number of history tokens supplied to Qwen.

²⁸gamma41/NORMAL ²⁹gamma2/PASSTROUGH ¹⁸gamma2/SOFT ¹⁹gamma2/FLAMINGO

HEc configuration	Tokens	Shandess Soft Prompt	Shandess HiCA
Initial ^{18,19}	1	78.02	78.80
Higher capacity ^{1,2}	2	79.74	80.08

Table 7: KT validation MeanAcc (%) without metadata or history Transformer. These historical configurations also differ in encoder capacity and validation configuration; their scores compare the complete configurations.

The two-token configurations obtain higher validation scores for both interfaces: 79.74% for Shandess Soft Prompt and 80.08% for Shandess HiCA. We retain two tokens as the reference for the subsequent metadata comparison. A separate token-count comparison with Transformer and metadata fixed is reported in Table 10.

5.4.2 Adding exercise metadata

Pedagogical metadata provide another view of the exercise: gameplay, module, objective and activity names, together with expert-written pedagogical intents. Their precomputed 1,024-dimensional Qwen3-Embedding-0.6B vectors enter HEc alongside exercise semantics and behavior, with a metadata-availability indicator. Both sides of this comparison use two output tokens per interaction and no history Transformer. No one-token, metadata-without-Transformer run was performed.

Metadata vectors are precomputed per exercise; missing metadata are zeroed.

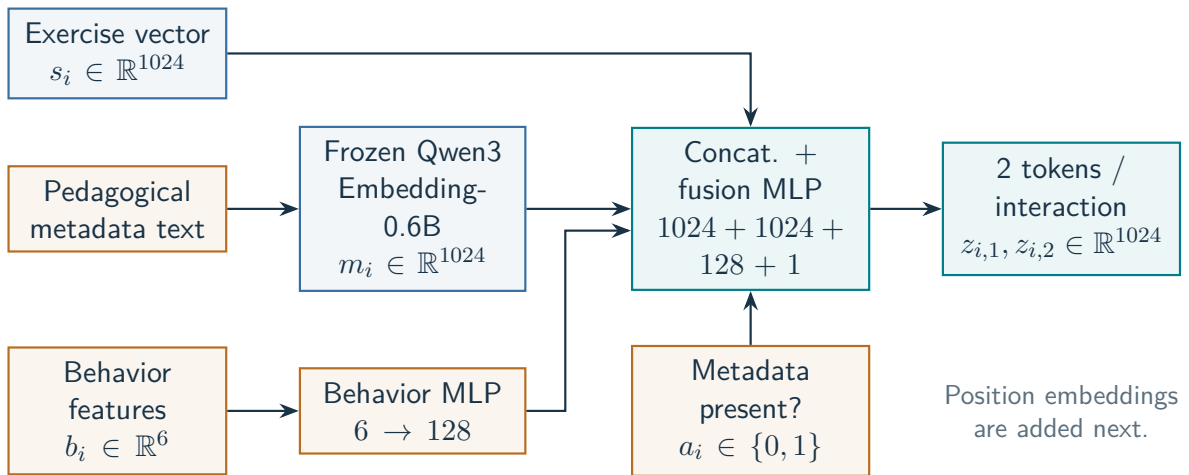


Figure 7: Pedagogical metadata augment exercise semantics and student behavior. The metadata comparison keeps two output tokens per interaction.

Two-token HEc	Shandess Soft Prompt	Shandess HiCA
Without metadata ^{1,2}	79.74	80.08
With metadata ^{20,23}	78.64	79.36

Table 8: KT validation MeanAcc (%), without Transformer and on shared validation windows. Shandess HiCA scores summarize their respective training lineages; they do not isolate metadata from initialization and training history.

Adding metadata lowers the selected score by 1.10 points for Shandess Soft Prompt and 0.72 point for Shandess HiCA in these runs. Extra pedagogical input therefore does not automatically improve

¹⁸gamma2/SOFT ¹⁹gamma2/FLAMINGO ¹gamma6/SOFT ²gamma6/FLAMINGO ²⁰gamma3/SOFT ²³gamma4/FLAMINGO

prediction. Redundant information or a harder optimization problem are possible explanations, not demonstrated mechanisms. The next variant tests whether contextualizing the interaction tokens helps.

5.4.3 Contextualizing the history with a Transformer

Without a Transformer, HEc constructs every interaction token independently. The representation of an attempt contains its exercise, outcome, duration and position, but does not depend on the other attempts. The downstream language model can combine the tokens, but HEc itself cannot identify repeated errors, changes over time, or relations between exercises before the history is injected.

The optional two-layer Transformer encoder is placed after the interaction MLP. It uses hidden dimension 1,024, four attention heads and feed-forward dimension 4,096. It changes the token representations without pooling the history, preserving the direct correspondence between interactions and token groups while allowing each token to incorporate the complete selected history.

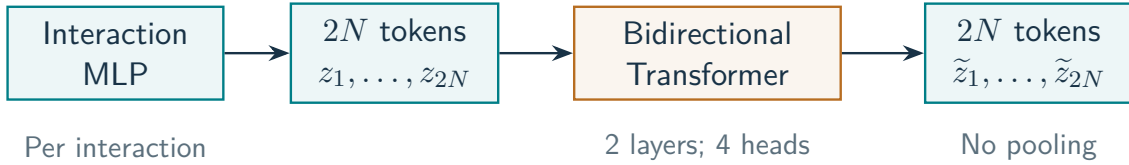


Figure 8: Optional bidirectional Transformer inside HEc. Self-attention contextualizes the historical interaction tokens before Shandess Soft Prompt or Shandess HiCA conditioning, without changing their number.

The Transformer is bidirectional rather than causal. All encoded interactions precede the prediction target and are known when the prediction is made. Allowing an older interaction to attend to a more recent observed interaction does not reveal the target response. A causal mask would impose an asymmetry that is not required when constructing a representation of this already observed history. The current exercise remains in the Qwen prompt; providing it to HEc for target-dependent retrieval is a separate extension.

The Shandess HiCA configurations with metadata use staged training: phase 1 trains the new cross-attention interface with frozen input components, while the subsequent continuation adapts selected components. The table distinguishes the interface phase from the results after subsequent training; initialization and freezing choices are described in Section 6.5.

Injection	Metadata	No Transformer	Transformer	Δ (pp)
Shandess HiCA	No	0.8008	0.7956	−0.52
Shandess Soft Prompt	No	0.7974	0.7968	−0.06
Shandess Soft Prompt	Yes	0.7864	0.7940	+0.76
Shandess HiCA, phase 1	Yes	0.7885	0.7916	+0.31
Shandess HiCA, after continuation	Yes	0.7936	0.7958	+0.22

Table 9: Validation results with and without the HEc Transformer on a shared validation set, with two tokens per interaction. Shandess HiCA rows include distinct phase and initialization lineages. Differences are in percentage points (pp); bold compares the two scores within each row.

Without metadata, Shandess HiCA + Transformer reaches 79.56% at step 6200, versus 80.08% for the Shandess HiCA h10.

With metadata, the Transformer improves Shandess Soft Prompt validation MeanAcc by 0.76 point, and the historical Shandess HiCA comparisons also improve. Without metadata, Shandess Soft Prompt remains close to its previous score (79.68% versus 79.74%). These experiments distinguish the

benefit of contextualizing the metadata-containing inputs from the cost of making this component part of every HEC.

One possible architectural explanation for the smaller historical Shandess HiCA gain is that repeated cross-attention already lets Qwen combine evidence across several history tokens. A Transformer in HEC supplies this interaction before injection and could therefore help Shandess Soft Prompt more. This explanation is consistent with the metadata-containing comparison, but not a general empirical rule, and has not been established by a mechanistic analysis.

At a hidden dimension of 1,024, the two-layer Transformer adds approximately 25.2 million parameters. This overhead belongs only to the variants that enable it; it must not be charged to the Shandess Soft Prompt or Shandess HiCA h10. The frozen semantic encoders are used offline and are separate from the online trainable history-conditioning modules.

Shandess Soft Prompt + Transformer + metadata	One token ⁷	Two tokens ⁵
Validation MeanAcc (%)	77.63	79.40
Test MeanAcc (%)	75.95	77.90

Table 10: Number of tokens with Transformer and metadata fixed. Comparisons are made within a row: validation with validation, test with test; bold marks the higher score within each row.

The two-token representation also improves over one token when metadata and the Transformer are both enabled: the validation gain is 1.77 points, and the common-test gain is 1.95 points. This supports allocating more than one token to an interaction in the architecture with metadata and a Transformer.

5.4.4 LoRA versus DoRA

A separate question is how to adapt Qwen. DoRA decomposes each adapted weight into magnitude and direction and applies a low-rank update to the directional component [8]. Separately learning the magnitude gives the adaptation additional flexibility without requiring a full-rank directional update, motivating its evaluation as an alternative to LoRA. We compare it with LoRA in Passthrough mode, which removes HEC and Shandess HiCA optimization from this comparison.

Adaptation	Best step	MeanAcc	Acc ₀	Acc ₁
DoRA ⁹	5800	0.7683	0.7912	0.7454
LoRA ⁸	4900	0.7745	0.7880	0.7610

Table 11: Best observed Passthrough validation results on the same validation set. Architecture and validation population are aligned; optimization budgets and checkpoint trajectories are not matched.

LoRA leads by 0.62 percentage point in this comparison. The earlier LoRA score of 0.7712 belongs to the earlier Passthrough run²⁹ under the historical validation configuration. The current evidence favors keeping LoRA, but does not establish a general superiority over DoRA: the training trajectories and code versions differ, and a different DoRA schedule could change the outcome. All configurations retained in the following sections use LoRA, so the adaptation method is omitted from their names except in explicit comparisons.

5.5 Final Shandess architecture

The successive experiments lead to a HEC with two tokens per interaction, without mandatory metadata or history Transformer. This configuration combines precomputed exercise semantics with the student’s

⁷gamma5/SOFT ⁵gamma42/SOFT ⁹DDPshandess/PASSTROUGH ⁸DDPshandess2/PASSTROUGH ²⁹gamma2/PASSTROUGH

observed behavior and retains the separation between history representation and history injection. Qwen2.5-0.5B-Instruct receives the current exercise as text and is adapted with rank-16 LoRA.

Shandess HiCA obtains the highest validation score, while Shandess Soft Prompt provides a lighter alternative with a small performance gap. The variants with metadata or a HEc Transformer clarify which additions help in individual comparisons, but do not displace this HEc configuration for the final accuracy–cost comparison.

Configuration	Val. MeanAcc (%)	Total params	Trainable params
Passthrough ⁸	77.45	502.8 M	8.8 M
Shandess Soft Prompt h10 ¹	79.74	515.1 M	21.0 M
Shandess HiCA h10 ²	80.08	572.9 M	78.9 M
Shandess Soft Prompt + T + M ⁵	79.40	542.3 M	48.3 M

Table 12: Selected configurations and parameter counts from their training logs. T denotes the HEc Transformer and M metadata. Offline semantic encoders are excluded. Shandess Soft Prompt/Shandess HiCA architectural comparisons use their common validation set; Passthrough retains its own validation configuration.

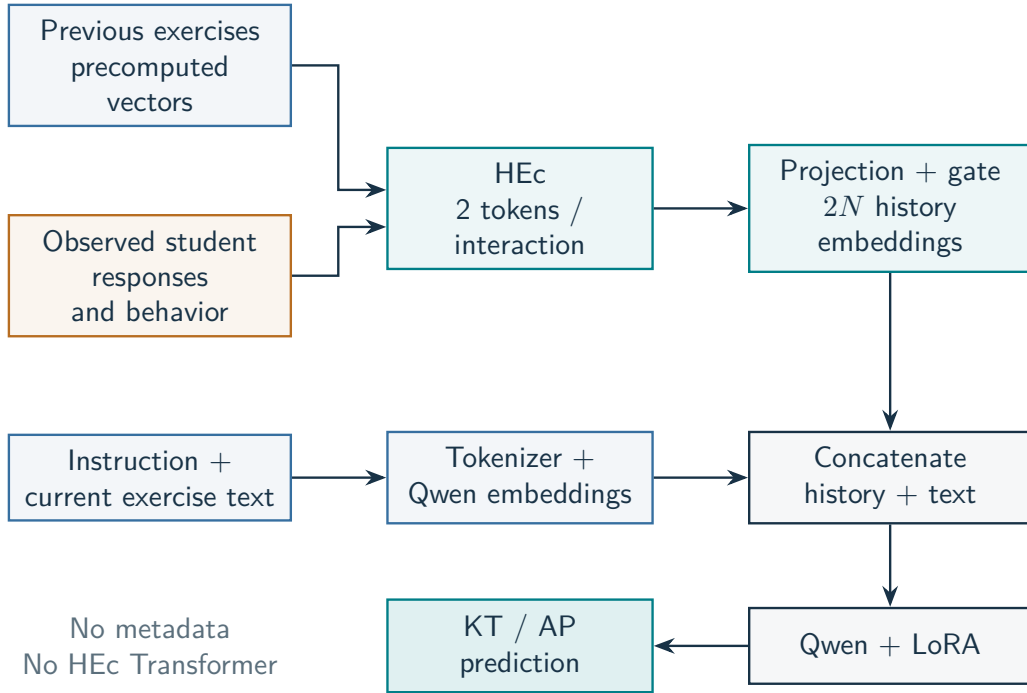


Figure 9: Final Shandess Soft Prompt configuration: two history tokens per interaction are projected and prepended to the current-exercise embeddings.

⁸DDPshandess2/PASSTHROUGH ¹gamma6/SOFT ²gamma6/FLAMINGO ⁵gamma42/SOFT

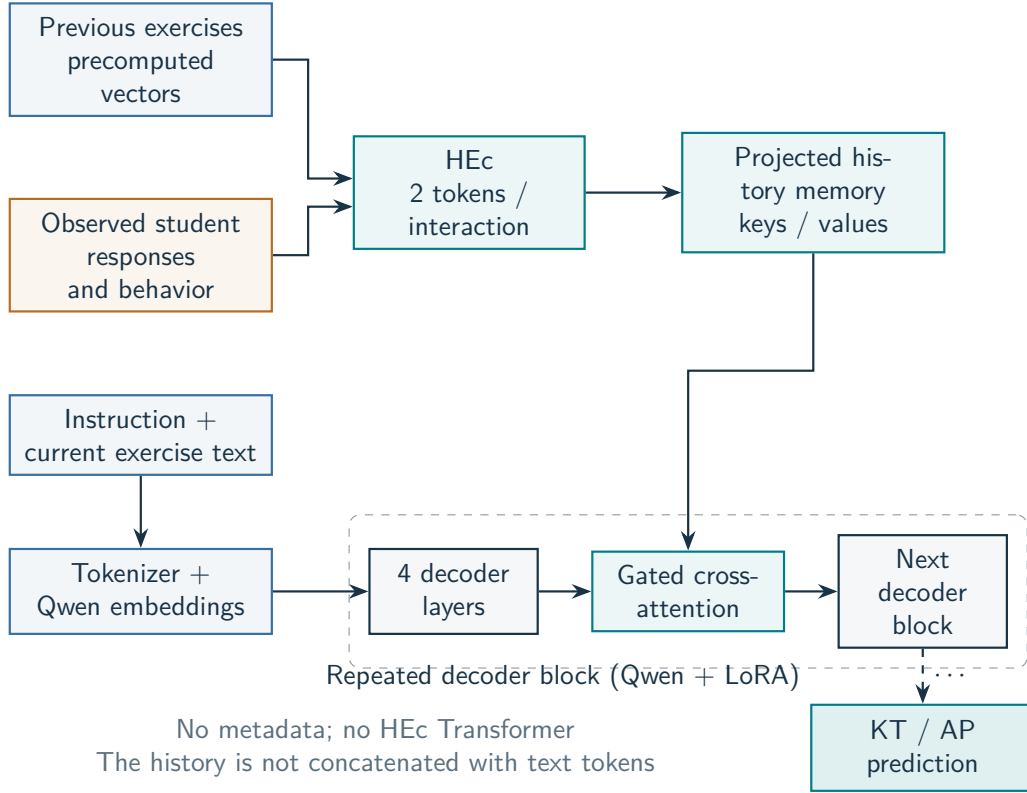


Figure 10: Final Shandess HiCA configuration: projected history tokens remain an external memory queried through six gated cross-attention blocks, one after each group of four Qwen decoder layers.

Both variants use a shared population model and semantic exercise inputs, without student-specific learned parameters. They process up to 200 preceding interactions using compact vectors instead of complete historical exercise texts. Shandess HiCA adds about 57.9 million parameters relative to Shandess Soft Prompt; the final evaluation measures the predictive gain and inference cost associated with this repeated access to history. Training lineages are described next, before the test comparisons and evaluation of the intended-use requirements.

6 Training Shandess

Section 5 establishes the history representation and the two conditioning mechanisms. Training must then make the predictor use these inputs to model student responses, including errors. This section specifies the data, objectives and checkpoints used in the main comparison; the supplementary training experiments are retained in Appendix A.

6.1 Dataset splitting and prediction windows

The preprocessed data contain 5,044,292 training interactions from 30,816 students and 1,221,743 test interactions from 7,704 different students. A further student-level split within the training table supplies validation targets. Consequently, neither validation nor test students contribute training windows. The main validation sets contain 10,000 targets and the common KT test contains 4,000 targets from 2,363 students. Earlier observations from a held-out student remain available as prediction context.

An eligible target at position t defines a window

$$W_t = (H_t, x_t, y_t), \quad H_t = ((x_i, r_i))_{i=\max(0, t-H)}^{t-1}, \quad (10)$$

where x_t contains the current exercise and information available before the student responds, y_t is the prediction target, and H is the maximum history length. Each earlier interaction (x_i, r_i) includes the exercise context and the observed response information r_i , including correctness, response duration and the selected option when available. These past outcomes are known at prediction time; the target response y_t is excluded from the input. The pair consisting of a student identifier and a target position identifies a window. Selection of unique windows is without replacement; the history preserves the observed chronological sequence.

The stored field `history_len`, denoted N , is the number of preceding interactions in this window, excluding the target. Thus $N = 1$ means one previous interaction and a distinct current target. It is measured before model-specific truncation and padding. The common test allows $1 \leq N \leq 200$; Shandess Soft Prompt may consume fewer interactions if its context limit requires truncation. Passthrough consumes no history even though each evaluated window still has a stored N . This distinction matters for both accuracy and latency curves in Section 7.4.

KT targets require textual exercise content and a valid correctness label. AP targets additionally require a multiple-choice attempt with a decodable selected option. For both tasks, the history may include other exercise formats.

6.2 KT and AP objectives

The same causal language model is used for both tasks, but the supervised target differs. Prompt tokens are masked in the labels, so the loss is applied only to the expected answer tokens.

For KT, the target is one token, z_0 or z_1 , representing an incorrect or correct response. Both strings map to distinct single tokens. The loss for one window is

$$\mathcal{L}_{\text{KT}}(W_t) = -\log p_{\theta}(z_{c_t} \mid x_t, H_t). \quad (11)$$

The implementation uses Qwen’s causal language-model cross-entropy over its vocabulary. At evaluation, the logits associated with the two response tokens provide a probability of success. This score-based evaluation is distinct from parsing an unrestricted language-model continuation.

For AP, the target is the textual index of the option selected by the student, followed by the end-of-sequence token. Writing the target token sequence as $a_t = (a_{t,1}, \dots, a_{t,m})$, the loss is

$$\mathcal{L}_{\text{AP}}(W_t) = -\frac{1}{m} \sum_{j=1}^m \log p_{\theta}(a_{t,j} \mid x_t, H_t, a_{t,<j}). \quad (12)$$

The model learns the student’s selected option, not the option marked correct in the exercise. The AP prompt specifies the number of candidate options, and predicted indices outside this range count as invalid predictions.

The experiments reported here train and evaluate each task separately. No KT-to-AP or AP-to-KT transfer experiment is reported; sharing the model components does not establish transfer of the learned history representation.

6.3 Target sampling and checkpoint selection

In MIAAM, 78.67% of interactions are correct. A predictor can therefore obtain high raw KT accuracy by always predicting success. In AP, always returning the correct option similarly agrees with most students without identifying the distractors selected by those who fail. We balance correct and incorrect

training targets in the principal configurations and monitor Acc_0 , Acc_1 and their average MeanAcc. In AP, Acc_0 measures the exact wrong option selected, rather than simply detecting a failure.

The AP validation experiments illustrate why this matters: natural target sampling yields 13.42% Acc_0 and 95.34% Acc_1 , whereas a 50% incorrect-target quota yields 53.70% and 79.72%. The balanced score rises from 54.38% to 66.71%. Increasing the quota to 65% further favors errors but lowers MeanAcc in the available runs. The full trajectories and training budgets are given in Appendix A. These observations motivate balanced learning, without establishing a universally optimal quota.

The AM/MIA target mixture is a separate sampling choice. The principal Shandess Soft Prompt adaptation uses equal AM and MIA target quotas; the natural-mixture adaptation remains a supplementary experiment. These quotas apply to prediction targets and do not reorder or rebalance a student’s chronological history. Checkpoint selection uses validation MeanAcc, not test accuracy. The available validation evidence and the exact evaluated checkpoints are recorded in Appendix D; a test checkpoint is not retrospectively labelled a validation optimum when its corresponding validation record is unavailable. In particular, the existing Shandess HiCA h1 tests use step 6900 (validation MeanAcc 0.8044), while the highest recovered validation MeanAcc is 0.8047 at step 5500. We retain the evaluated checkpoint so that the reported comparisons remain consistent.

6.4 History thresholds and the evaluated models

The suffix **h10** denotes a minimum of ten preceding interactions admitted during training; **h1** denotes a minimum of one. Neither suffix fixes the history length: the principal HEc configurations admit up to 200 interactions. The notation describes training eligibility, whereas N denotes the history supplied to a prediction window at evaluation.

The initial architecture comparisons used h10. This excludes training targets with only one to nine preceding interactions. The subsequent h1 experiments admit these short histories, allowing the model to encounter the limited-context situations examined in the student cold-start analysis. The value ten is the threshold of the initial experiments, not an established optimal minimum. We retain h10 results in the supplementary comparisons.

The main KT evaluation follows two fixed checkpoints throughout accuracy, latency, probability quality and cold start:

Model	Minimum history	Checkpoint step
Shandess HiCA h1 ³	1	6900
Shandess Soft Prompt h1 ¹²	1	4000

Table 13: Models used in the main KT comparison. Both use two HEc tokens per interaction, without metadata or a HEc Transformer. Shandess Soft Prompt h1 denotes the AM/MIA 50/50 adaptation throughout the main evaluation.

This selection keeps one model per conditioning method and includes short histories in training. It is not a claim that these runs maximize every test metric. In particular, the alternative natural-mixture Shandess Soft Prompt h1 checkpoint is preserved in Appendix B.

6.5 Initialization and computational setup

Shandess trains HEc, its projection and Qwen LoRA, together with the gated cross-attention modules for Shandess HiCA. Reusing learned history components allows the conditioning interface to be studied without repeating all earlier training. Initialization nevertheless differs between runs and must be kept separate from their architectural description.

³gamma8/FLAMINGO ¹²gamma1111/SOFT

The selected Shandess Soft Prompt h1 checkpoint continues the complete Shandess Soft Prompt h10 checkpoint at step 6600, with a new initial learning rate of 10^{-6} . Its step 4000 is local to this adaptation. The h10 Shandess HiCA experiment instead reuses HEc and its projection from that Shandess Soft Prompt checkpoint while training new Qwen LoRA and cross-attention parameters. The documented component histories, including the older staged experiments, are retained in Appendix A. Differences between h1 and h10 checkpoints therefore cannot automatically be attributed to the history threshold alone.

Training uses H100 GPUs with 80 GB of memory on Jean Zay, with bfloat16 and gradient checkpointing in the recent runs. The documented two-token h10 runs and Shandess Soft Prompt adaptations use four GPUs, a per-device batch of 20 and a global batch of 80. Initial training typically starts at 10^{-5} ; the adaptation above starts at 10^{-6} . Run-specific schedules, validation trajectories and resource records are provided in the appendices. Training cost includes prerequisite components when they are reused; an older Transformer-based pipeline’s cost is not assigned to the selected Shandess HiCA h1 checkpoint.

The local recurrent baselines use the pyKT implementations: DKT predicts at the activity level with hidden dimension 200, while qDKT-ID and qDKT-text predict at the exercise level with dimensions 300 and 384. For qDKT-text, MiniLM vectors initialize the response-dependent embedding table, which is then learned during training; the sentence encoder itself is frozen and used offline. The local LKT implementation uses CamemBERT-base with a 512-token input budget.

DKT and both qDKT baselines were trained using 50,000 windows each. The Shandess training pools listed in Table 17 contain more than one million windows, although pool size does not imply that every window was visited before the selected checkpoint. Training sample sizes and optimization budgets are therefore not matched across model families. The common test compares the resulting trained systems; it does not isolate the contribution of architecture at a fixed training budget.

The final evaluation uses fixed weights. Predictions are aligned by target identity before computing metrics. The common balanced KT test is distinct from the fixed-target history experiment and the new-exercise cohort; their population counts and sampling details are retained in Table 42.

7 Final Evaluation and Baseline Comparison

The evaluation asks whether the models retained in Section 6.4 improve prediction over the local baselines, at what inference cost, and with what quality of success probabilities. Shandess HiCA h1³ and Shandess Soft Prompt h1¹² refer to the same checkpoints throughout this section; the latter uses AM/MIA 50/50 adaptation. Complete results for the other configurations remain in Appendix B.

7.1 Knowledge Tracing: predictive gain over the baselines

The common KT test contains 4,000 aligned targets from 2,363 held-out students, with 2,000 correct and 2,000 incorrect responses. It admits histories of one to 200 interactions and imposes no AM/MIA quota: the sample contains 2,596 AM and 1,404 MIA targets. Accuracy therefore equals MeanAcc on this test. All local models receive the same target identities and labels; each uses its own input representation and context budget. The regenerated cache for Normal and the Shandess Soft Prompt adaptations was checked for target-level alignment, although the export does not establish byte-identical historical inputs. Detailed protocol evidence accompanies the full results.

³gamma8/FLAMINGO ¹²gamma1111/SOFT

Model	Acc. = MeanAcc	Acc ₀	Acc ₁	AUC
Shandess HiCA h1	78.375	80.550	76.200	0.8613
Shandess Soft Prompt h1	77.625	78.450	76.800	0.8560
DKT	75.975	74.500	77.450	0.8340
qDKT-ID	74.200	73.450	74.950	0.8161
qDKT-text	74.100	73.200	75.000	0.8177
LKT	75.750	77.000	74.500	0.8342
Passthrough	75.350	77.500	73.200	0.8194
Normal	75.875	77.000	74.750	0.8335

Table 14: Common KT test: 4,000 targets, 2,000 per class. Accuracy columns are percentages. The two Shandess checkpoints are fixed in Table 13; the full configuration table is in Appendix B.

Both Shandess models exceed every locally evaluated baseline in aggregate KT accuracy. Relative to DKT, the strongest baseline on this test, the gains are 2.40 percentage points for Shandess HiCA and 1.65 for Shandess Soft Prompt. The improvement over Passthrough shows that observed student history adds predictive information beyond the current exercise. Normal’s smaller gain over that control also indicates that providing history as text is not sufficient to obtain the same benefit under the evaluated context budget. These compare complete trained systems, rather than isolating causal effects of their conditioning mechanisms.

The AUC results also indicate better overall discrimination. However, the 0.75-point difference between the two Shandess checkpoints is smaller than either gain over DKT. Their ordering remains a point estimate, not a settled ranking across training runs. The available paired uncertainty analyses are in Appendix C; intervals involving an h10 checkpoint are not transferred to h1. Conditional AM/MIA results in Table 26 complement the aggregate comparison.

Relation to published MIAAM results. The MIAAM benchmark[2] reports an approximate MeanAcc of 78% for Gemma with exercise text and pedagogical descriptions. Shandess reaches a similar range with a predictor below one billion parameters. The published targets, source mixture and history budgets differ from the local protocol; Table 27 therefore presents these reference scores separately.

High overall accuracy nevertheless hides a marked imbalance. With exercise text, Mistral Small 4 reaches 82% accuracy but 97% Acc₁ and 33% Acc₀; even Gemma with pedagogical descriptions reaches 86% overall with 91% versus 65%. These models identify successful responses much more reliably than failures. Weak failure detection limits their usefulness, as evaluated, for personalized trajectories that adapt exercise selection to students’ difficulties.

7.2 Inference cost of the predictive gain

For repeated exercise scoring, the question is how much computation the additional predictive information requires. We compare median model-call latency at batch size one, including HEc for Shandess. Input preparation, device transfers and offline exercise embedding are excluded. Five warm-up calls precede twenty synchronized measurements at each stored history length. Full measurements, precision settings and LKT variability are retained in Appendix B.

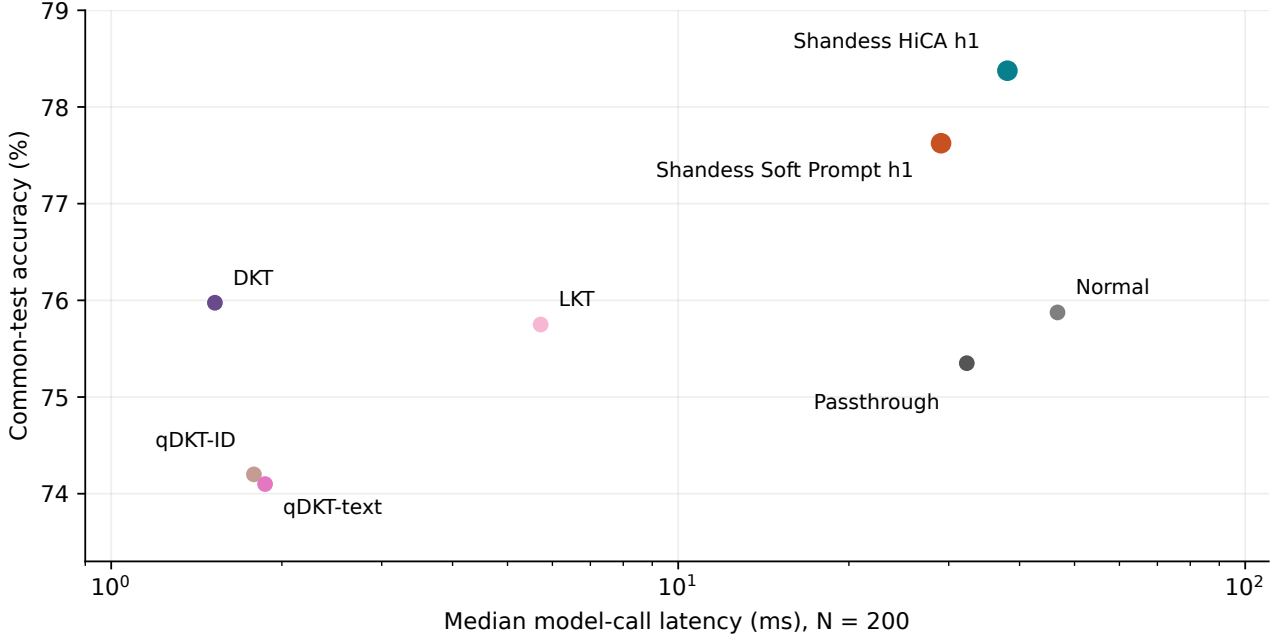


Figure 11: Common-test KT accuracy against median model-call latency at stored history length 200, batch size one. The horizontal axis is logarithmic. The selected checkpoints are those of Table 14. Timings come from separate jobs and exclude input preparation.

Shandess improves predictive quality at a substantial cost relative to recurrent KT: DKT takes about 1.52 ms per measured call, versus 29.08 ms for Shandess Soft Prompt and 38.07 ms for Shandess HiCA. Their calls are thus approximately 19 and 25 times as long in these measurements. Shandess Soft Prompt offers the lower measured inference cost of the two while retaining most of their gain over DKT. Normal is slower than either selected Shandess model, so textual history is not a cheaper substitute in this workload.

The baseline benchmark replays the supplied history rather than reusing an incremental recurrent state. HEc also pads to its configured maximum length; nearly flat latency curves do not establish constant complexity in history length. LKT has substantial variability: its median is 5.72 ms but its mean is 21.02 ms at length 200. These qualifications matter when estimating a deployment budget from the medians. Reusing HEc outputs across candidate exercises could reduce repeated scoring work, but neither complete selection-policy latency nor end-to-end serving cost has been measured.

7.3 Quality of the predicted probabilities

Selecting suitable exercise difficulty requires a meaningful success probability, not only a correct binary decision. We compare Brier score, $n^{-1} \sum_i (p_i - c_i)^2$, and expected calibration error using ten fixed equal-width probability bins (ECE_{10}). Brier reflects discrimination and calibration together; ECE measures average disagreement between predicted and observed success frequencies within the chosen bins.

Model	Brier	ECE ₁₀
Shandess HiCA h1	0.1509	0.0128
Shandess Soft Prompt h1	0.1541	0.0149
DKT	0.1656	0.0186
qDKT-ID	0.1765	0.0361
qDKT-text	0.1751	0.0315
LKT	0.1690	0.0550
Passthrough	0.1729	0.0209
Normal	0.1662	0.0196

Table 15: Probability metrics on the common balanced KT test. Lower is better; no post-hoc calibration is fitted.

Both Shandess checkpoints improve on DKT in Brier score and ECE, so their benefit extends beyond the default classification threshold. Reliability curves give a more direct view of agreement across probability ranges.

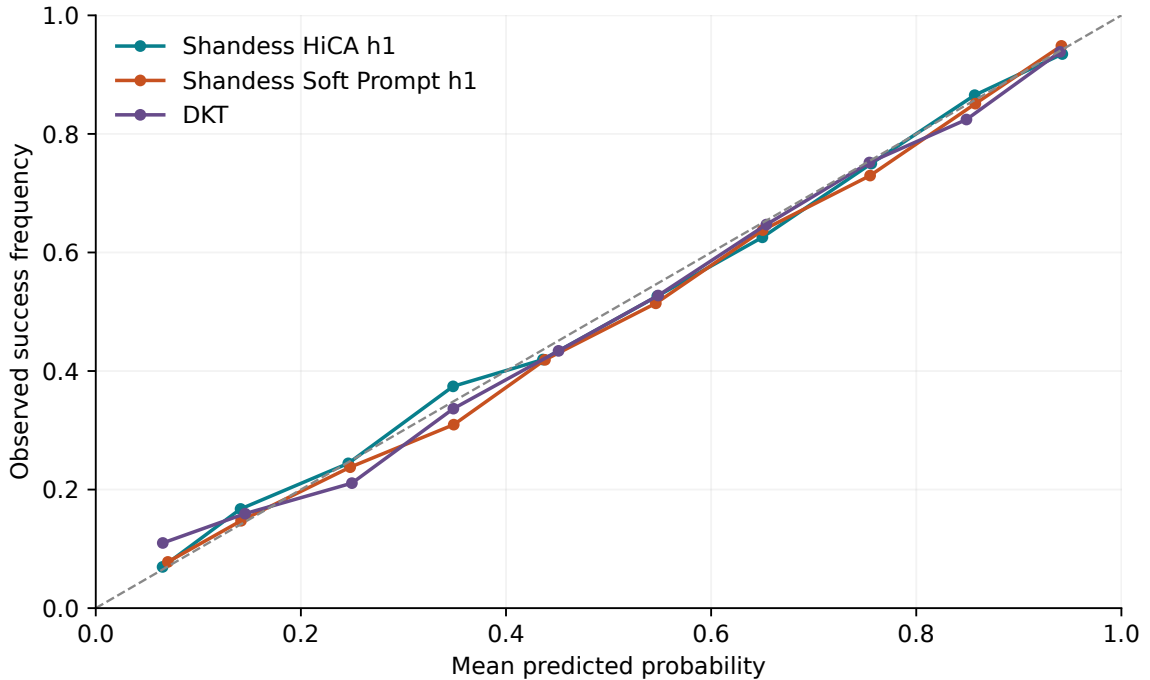


Figure 12: Reliability of the selected Shandess models and DKT on the balanced KT test. The diagonal denotes agreement between predicted and observed success probabilities. Ten fixed equal-width bins are used. The full Brier/ECE table is in Appendix B; all curves and bin counts remain in the repository.

These measurements support using the probabilities in a difficulty criterion, but do not establish calibration under the natural response distribution. The test is deliberately balanced, whereas most MIAAM responses are correct. ECE also depends on binning and sample size. Calibration on representative deployment data and evaluation within an exercise-selection policy remain necessary; no calibration model is fitted on this test.

7.4 New students and limited history

Naturally short histories. The test students are absent from training, so personalization must use their observed interactions. The 266 common-test targets with one to nine preceding interactions show

prediction with little available history.

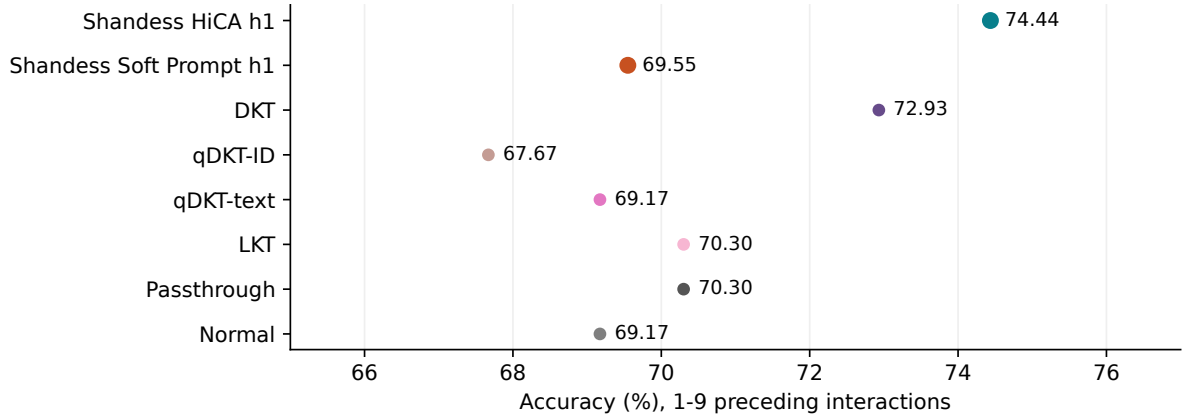


Figure 13: Accuracy on the same 266 targets with one to nine preceding interactions, representing 264 held-out students. This subgroup is not balanced by correctness, so accuracy is not necessarily MeanAcc. No zero-history condition is evaluated.

The aggregate advantage does not hold uniformly in this subgroup. Shandess HiCA has the highest point estimate, while DKT remains close and Shandess Soft Prompt falls below it. The two methods thus have different practical limitations when few observations are available. The subgroup is small and does not establish a general ranking of adaptation speed. Full history-group curves and h1/h10 comparisons are retained in Appendix E.

Additional history for the same prediction. A second experiment fixes 4,000 targets from 1,075 held-out students with at least 200 preceding interactions. Each model receives successively longer suffixes of history, from one to 200 interactions, without changing the target or its weights. This separates context availability from changes in the target population.

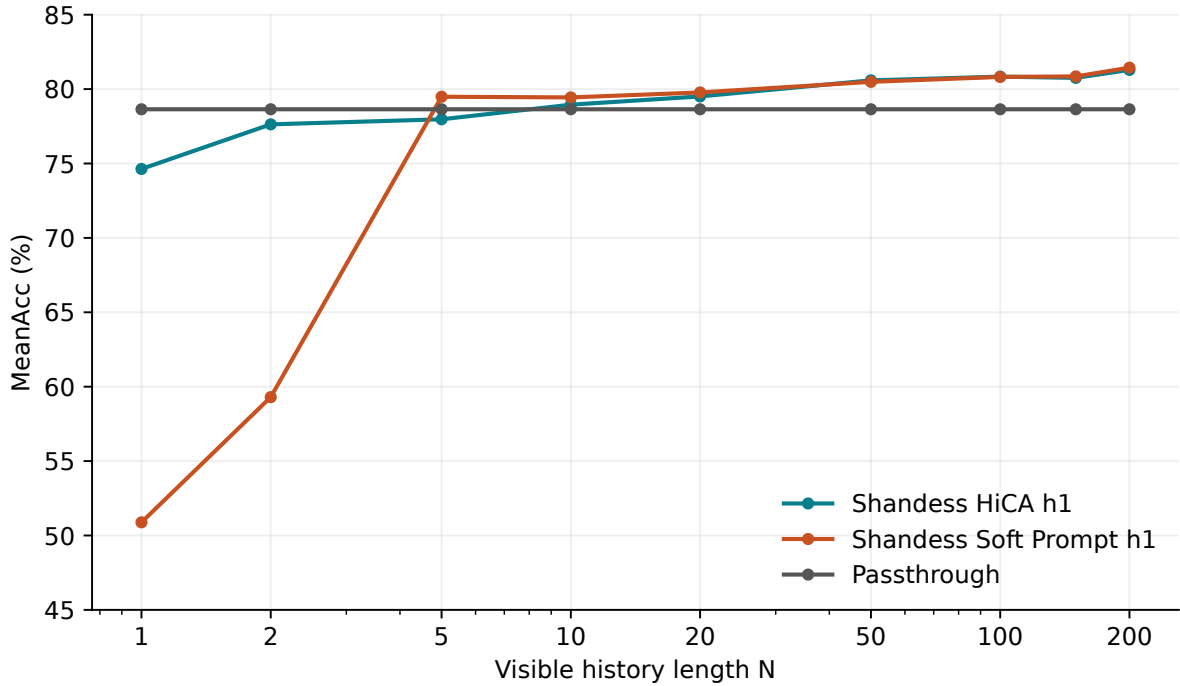


Figure 14: MeanAcc on fixed targets as visible history increases. The selected Shandess checkpoints are compared with the history-independent Passthrough control. All nine evaluated lengths are shown on a logarithmic axis. The targets come from long trajectories; truncation does not reproduce the students’ actual first interactions.

With one or two observations, Shandess Soft Prompt remains close to chance in MeanAcc despite its h1 adaptation, then improves markedly with more context. Shandess HiCA handles the earliest points better, although it too benefits from additional observations. With longer histories, both exploit information unavailable to Passthrough and approach similar balanced scores. The benefit is not strictly monotone at every length.

These curves identify where each predictor is useful; they do not isolate whether its behavior originates in architecture, initialization or training exposure. Accuracy, conditional metrics and paired intervals, together with the full six-checkpoint figure, are preserved in Appendix E. No corresponding fixed-target AP evaluation is available.

7.5 Prediction on new exercises

The new-exercise test contains 66 first-encounter targets from 54 AM exercises absent from the non-test data, contributed by 21 students. Histories retain previous interactions with other exercises. Prediction must therefore use exercise content and student experience without having trained on these exercise identities.

Model	Correct	Accuracy	MeanAcc
Shandess HiCA h1	59/66	89.39	93.27
Shandess Soft Prompt h1	57/66	86.36	91.35
Passthrough	34/66	51.52	56.18

Table 16: First encounters with test-only AM exercises: 66 targets from 54 exercises and 21 students. Percentages use the same targets for every model.

Both Shandess models substantially outperform Passthrough on this cohort, providing initial

evidence of useful prediction on new exercises within MIAAM. Their two-prediction difference does not support a reliable ranking. Nor does this experiment establish superiority over KT baselines that were not evaluated on these targets.

The sample is small and uneven: one student contributes 18 targets, and 43 exercises have only one target. Seen-exercise controls also differ in history length and student population, so higher scores on unseen targets must not be interpreted as a benefit of novelty. Complete seen/unseen results, conditional metrics and uncertainty remain in Appendix E.

7.6 Answer Prediction: predicting student errors

AP asks whether the predictor can identify the option a student selects, including a wrong answer. On the 4,000-target AP test, only 357 responses are incorrect. Always returning the correct exercise answer achieves 91.075% raw accuracy, while its Acc_0 is zero and its MeanAcc is 50%.

Shandess Soft Prompt trained with a 50% incorrect-target quota obtains 62.184% MeanAcc at the available test checkpoint, including 44.818% accuracy on exact wrong options. Its lower raw accuracy therefore accompanies a stronger ability to model student errors. The 65% quota further favors those errors but reduces the balanced score in the available runs.

These tests support analysis of student-error prediction, not a matched Shandess HiCA versus Shandess Soft Prompt comparison in AP. The checkpoints have different training budgets, and the tested 50% checkpoint precedes its later validation improvement. Full results, published MIAAM references and source-by-option breakdowns are in Appendix B. Larger answer sets make exact distractor prediction harder and contribute to the aggregate AM/MIA gap; that composition must be considered before attributing the gap to the platform alone. No cross-task transfer result is established.

7.7 Does Shandess meet the intended-use requirements?

The common KT comparison supports a predictive benefit over local baselines, together with improved probability scores. That benefit requires more computation than recurrent KT, making inference budget a relevant constraint on repeated exercise scoring.

Personalization for held-out students is supported when observations are available, with a clear limitation for Shandess Soft Prompt at very short contexts. New-exercise results provide preliminary within-MIAAM evidence; broader generalization, deployment calibration and the effect on actual learning trajectories remain to be evaluated. These are the priorities for the next stage of the work.

8 Future Work

This work focused on constructing a student model that can use long interaction histories and the content of previously unseen exercises while remaining small enough for practical use. The resulting architecture satisfies these constraints only partially. In particular, the prediction language model still dominates its size and inference cost, the experiments are restricted to MIAAM, and the model predicts student behavior and not which exercise should be presented next. These limitations define three main directions for future work.

8.1 Prediction model family, size and computational cost

Shandess currently uses Qwen2.5-0.5B-Instruct as its prediction model. Although this model is small relative to frontier language models, it still accounts for most of the parameters and computation of Shandess. This is particularly important for personalized sequencing. At each step, the same student may need to be evaluated on many candidate exercises, requiring the prediction model to be executed

once for every candidate. The inference cost can therefore become more restrictive than the training cost.

A first direction is to replace Qwen with a much smaller Transformer trained from scratch on the student-modeling tasks. The rest of the architecture, including HEc and initially the frozen exercise encoder, could remain unchanged. This would isolate the contribution of the pretrained prediction model rather than changing every representation at the same time. Specialized Transformers such as SAKT already show that self-attention models trained from scratch can effectively model student interaction histories [12]. The open question is whether such a model can also exploit the semantic representations used by Shandess well enough to replace a pretrained language model.

The comparison should not be restricted to aggregate predictive performance. LKT and NTKT suggest that pretrained language representations are particularly useful when the question content must be interpreted or when exercises are unseen during training [7, 11]. A smaller model could therefore match Shandess on the standard test distribution while losing its robustness to exercise cold start. Comparing accuracy, inference cost and performance on unseen exercises would quantify this trade-off. If the from-scratch Transformer remains competitive, Shandess could be made substantially lighter. If its performance degrades mainly under distribution shift, the experiment would instead identify the robustness provided by the pretrained language model and the price paid for it at inference time.

A complementary direction is to evaluate Shandess HiCA and Shandess Soft Prompt with other pretrained language-model families at comparable sizes, and with larger Qwen models. These comparisons would test whether the benefits of history conditioning generalize beyond Qwen2.5-0.5B-Instruct and how they change with model capacity. Using matched data splits and history settings, each model should be compared with its own Passthrough baseline to determine whether the gain from student history persists as size increases. Predictive performance, probability calibration and inference cost should be assessed jointly: a larger model would only be preferable for personalized sequencing if its gains justify the additional cost of evaluating candidate exercises.

8.2 Cross-dataset generalization and rapid adaptation

The present experiments train and evaluate Shandess only on MIAAM. The student split evaluates generalization to new students, and the small set of unseen test exercises provides an initial exercise cold-start setting, but neither tests whether the architecture transfers to a different educational platform. It is therefore not yet possible to distinguish properties of Shandess from regularities specific to Adaptiv’Math and MIA Seconde.

The cold-start results sharpen these limits. The exercise experiment contains only 66 targets from 21 students and 54 AM exercises, with longer histories than the seen-exercise controls. A larger exercise-held-out evaluation across more students, with explicit history and difficulty diagnostics, would test whether the observed advantage over Passthrough persists. Repeated training seeds and exercise-level uncertainty would complement the current exploratory student-bootstrap intervals. Such extensions are future experiments, not results supplied by the present evaluation.

The fixed-target curves also expose a specific short-history weakness: Shandess Soft Prompt remains unreliable with one or two visible interactions even after adaptation with a minimum history of one. Controlled experiments on short-history sampling and conditioning could distinguish a training-distribution issue from a limitation of the representation or its injection mechanism. A genuine zero-history evaluation would require an explicitly supported empty-history input; no such condition was measured here.

This question is becoming testable as new datasets expose the actual content of exercises rather than only identifiers and binary outcomes. FoundationalASSIST contains 1.7 million English-language interactions from 5,000 students, together with full question text, selected answers, distractors and alignments to Common Core standards [15]. XES3G5M provides 5.55 million mathematics interactions from 18,066 students and 7,652 questions, with textual question content, question analyses, question types and a structured set of knowledge components [9]. These datasets differ from MIAAM in language, educational context, exercise structure and available metadata, making them concrete tests of cross-dataset robustness.

The main objective would not simply be to retrain Shandess independently on each dataset. It would

be to measure how much of a model trained on one dataset can be reused on another and how quickly it can be adapted. Performance without adaptation would measure direct transfer, while fine-tuning with a controlled and progressively increasing amount of target data would measure adaptation speed. A model that reaches its target-dataset performance with few additional interactions would be more useful for a new educational platform than a model that must be trained again from scratch. Conversely, a large transfer gap would show that the current semantic and behavioral representations remain tied to the data distribution on which they were learned.

8.3 From student prediction to personalized trajectory construction

Shandess predicts whether a student will answer correctly and, for multiple-choice exercises, which answer they are likely to select. These predictions provide information about the student, but they do not by themselves define a learning trajectory. The final direction is therefore to use Shandess inside an exercise-selection mechanism that compares candidate activities and decides what the student should attempt next.

The selection objective cannot be reduced to choosing the exercise with the highest predicted probability of success. Such a policy would systematically favor exercises that the student has already mastered. Instead, the prediction should be combined with an objective related to learning. ZPDES formulates personalized activity selection as a bandit problem driven by estimated learning progress [3]. MAGELLAN similarly uses predictions of competence and learning progress to select goals in large and evolving spaces [5]. These approaches provide two natural starting points for turning Shandess from a response predictor into a component of a sequencing policy.

In such a system, Shandess could evaluate the expected behavior of a student on each candidate exercise, while the selection mechanism would favor exercises that are neither already mastered nor currently inaccessible. Answer Prediction could provide additional information by identifying likely distractors and therefore distinguishing different incorrect behaviors that have the same KT outcome. However, the resulting system would need to be evaluated on learning over complete trajectories, rather than only on the accuracy of the next-response prediction. Since the trajectories in MIAAM were generated by an existing adaptive policy, they cannot by themselves establish the causal effect of a new policy. A complete validation would ultimately require either a sufficiently reliable student simulator or a prospective evaluation with learners.

9 Conclusion

Shandess combines semantic exercise representations and student interaction histories through a dedicated encoder and two language-model conditioning mechanisms. The experiments support its usefulness for knowledge tracing within MIAAM, while identifying limitations in short-context prediction, inference cost and the scope of generalization. Integrating these predictors into exercise selection remains the next step toward evaluating their contribution to personalized learning trajectories.

References

- [1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob L. Menick, Sebastian Borgeaud, Andy Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikołaj Bińkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karén Simonyan. Flamingo: a visual language model for few-shot learning. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/960a172bc7fbf0177ccccbb411a7d800-Abstract-Conference.html.
- [2] Anonymous authors. MIAAM: a semantically rich multimodal education dataset of 7 million maths learning interactions that challenges frontier AI models. Anonymized manuscript submitted to NeurIPS 2026; supplied project version, 2026.
- [3] Benjamin Clement, Didier Roy, Pierre-Yves Oudeyer, and Manuel Lopes. Multi-armed bandits for intelligent tutoring systems. *Journal of Educational Data Mining*, 7(2):20–48, 2015. doi: 10.5281/zenodo.3554667. URL <https://jedm.educationaldatamining.org/index.php/JEDM/article/view/JEDM111>.
- [4] Albert T. Corbett and John R. Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4:253–278, 1994. doi: 10.1007/BF01099821. URL <https://doi.org/10.1007/BF01099821>.
- [5] Loris Gaven, Thomas Carta, Clément Romac, Cédric Colas, Sylvain Lamprier, Olivier Sigaud, and Pierre-Yves Oudeyer. MAGELLAN: Metacognitive predictions of learning progress guide autotelic LLM agents in large goal spaces. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 18920–18953. PMLR, 2025. URL <https://proceedings.mlr.press/v267/gaven25a.html>.
- [6] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- [7] Unggi Lee, Jiyeong Bae, Dohee Kim, Sookbun Lee, Jaekwon Park, Taekyung Ahn, Gunho Lee, Damji Stratton, and Hyeoncheol Kim. Language model can do knowledge tracing: Simple but effective method to integrate language model and knowledge tracing task. arXiv:2406.02893v2, 2024. URL <https://arxiv.org/abs/2406.02893v2>.
- [8] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. DoRA: Weight-decomposed low-rank adaptation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32100–32121. PMLR, 2024. URL <https://proceedings.mlr.press/v235/liu24bn.html>.
- [9] Zitao Liu, Qiongqiong Liu, Teng Guo, Jiahao Chen, Shuyan Huang, Xiangyu Zhao, Jiliang Tang, Weiqi Luo, and Jian Weng. XES3G5M: A knowledge tracing benchmark dataset with auxiliary information. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/67fc628f17c2ad53621fb961c6bafcaf-Abstract-Datasets_and_Benchmarks.html.
- [10] Hiromi Nakagawa, Yusuke Iwasawa, and Yutaka Matsuo. Graph-based knowledge tracing: Modeling student proficiency using graph neural network. In *2019 IEEE/WIC/ACM International Conference on Web Intelligence*, pages 156–163, 2019. doi: 10.1145/3350546.3352513. URL <https://doi.org/10.1145/3350546.3352513>.

- [11] Max Norris, Kobi Gal, and Sahan Bulathwela. Next token knowledge tracing: Exploiting pretrained LLM representations to decode student behaviour. arXiv:2511.02599v2, 2026. URL <https://arxiv.org/abs/2511.02599v2>. Version 2, 26 January 2026; first submitted November 2025.
- [12] Shalini Pandey and George Karypis. A self-attentive model for knowledge tracing. In *Proceedings of the 12th International Conference on Educational Data Mining*, pages 384–389, 2019. URL <https://files.eric.ed.gov/fulltext/ED599186.pdf>.
- [13] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. In *Advances in Neural Information Processing Systems*, volume 28, 2015. URL <https://papers.nips.cc/paper/5654-deep-knowledge-tracing>.
- [14] Shashank Sonkar, Andrew E. Waters, Andrew S. Lan, Phillip J. Grimaldi, and Richard G. Baraniuk. qDKT: Question-centric deep knowledge tracing. In *Proceedings of the 13th International Conference on Educational Data Mining*, pages 677–681, 2020. URL <https://people.umass.edu/~andrewlan/papers/20edm-qdkt.pdf>.
- [15] Eamon Worden, Cristina Heffernan, Neil Heffernan, and Shashank Sonkar. FoundationalASSIST: An educational dataset for foundational knowledge tracing and pedagogical grounding of LLMs. arXiv:2602.00070, 2026. URL <https://arxiv.org/abs/2602.00070>.

A Training details and supplementary experiments

The following records supplement the training protocol in Section 6 with window counts, sampling comparisons, initializations and resource accounting.

A.1 Window populations

Split / configuration	Source	Actual windows	Role
Train, Passthrough ⁸	Train users	1,528,641	Learning
Train, Shandess Soft Prompt/Shandess HiCA h10 ^{1,2}	Train users	1,381,126	Learning
Train, Shandess Soft Prompt h1 (natural) ¹¹	Train users	1,906,810	Adaptation
Train, Shandess Soft Prompt h1 (50/50) AM/MIA ¹²	Train users	1,528,641	Adaptation
Validation	Held-out train-table users	10,000	Selection
Common KT test	Test-table users	4,000	Evaluation

Table 17: Actual window counts recorded in the logs for selected configurations. With `maximize-windows`, the requested count in a cache name (for example `train60000`) is not the loaded count. Target quotas and eligibility determine how many unique windows can be retained. Student counts in the source tables are not sample counts.

Quotas apply to prediction targets; their histories remain chronological. Validation-cache identities are listed in Appendix D, and the cold-start cohorts in Table 42.

A.2 Correctness sampling in Answer Prediction

The historical AP comparison below motivated the balancing procedure in Section 6.3: its ranking by overall accuracy favors Shandess HiCA, whereas its ranking by MeanAcc favors Passthrough. For AP, Acc_0 measures prediction of the exact incorrect option selected by the student, not merely detection of failure.

Historical AP model	Accuracy	Acc_0	Acc_1	MeanAcc
Passthrough	85.18	43.26	88.81	66.04
Shandess Soft Prompt	86.98	31.97	91.74	61.86
Shandess HiCA	88.98	31.03	94.00	62.52

Table 18: Historical AP tests (%), under the original imbalanced protocol. The highest raw accuracy and highest MeanAcc belong to different models.

The later quota comparison uses Shandess Soft Prompt with two HEC tokens, no Transformer or metadata, and equal AM/MIA target quotas. Training budgets and selected steps differ across runs.

Incorrect-target quota	Best step	Acc_0	Acc_1	MeanAcc
Natural ¹⁴	900	13.42	95.34	54.38
50% ¹³	4800	53.70	79.72	66.71
65% ¹⁵	7900	66.52	48.54	57.53

Table 19: AP validation accuracies (%) at the selected checkpoints. Training budgets and selected steps belong to the respective runs.

⁸DDPshandess2/PASSTHROUGH ¹gamma6/SOFT ²gamma6/FLAMINGO ¹¹gamma111/SOFT ¹²gamma1111/SOFT ¹⁴gamma92/SOFT
¹³gamma91/SOFT ¹⁵gamma94/SOFT

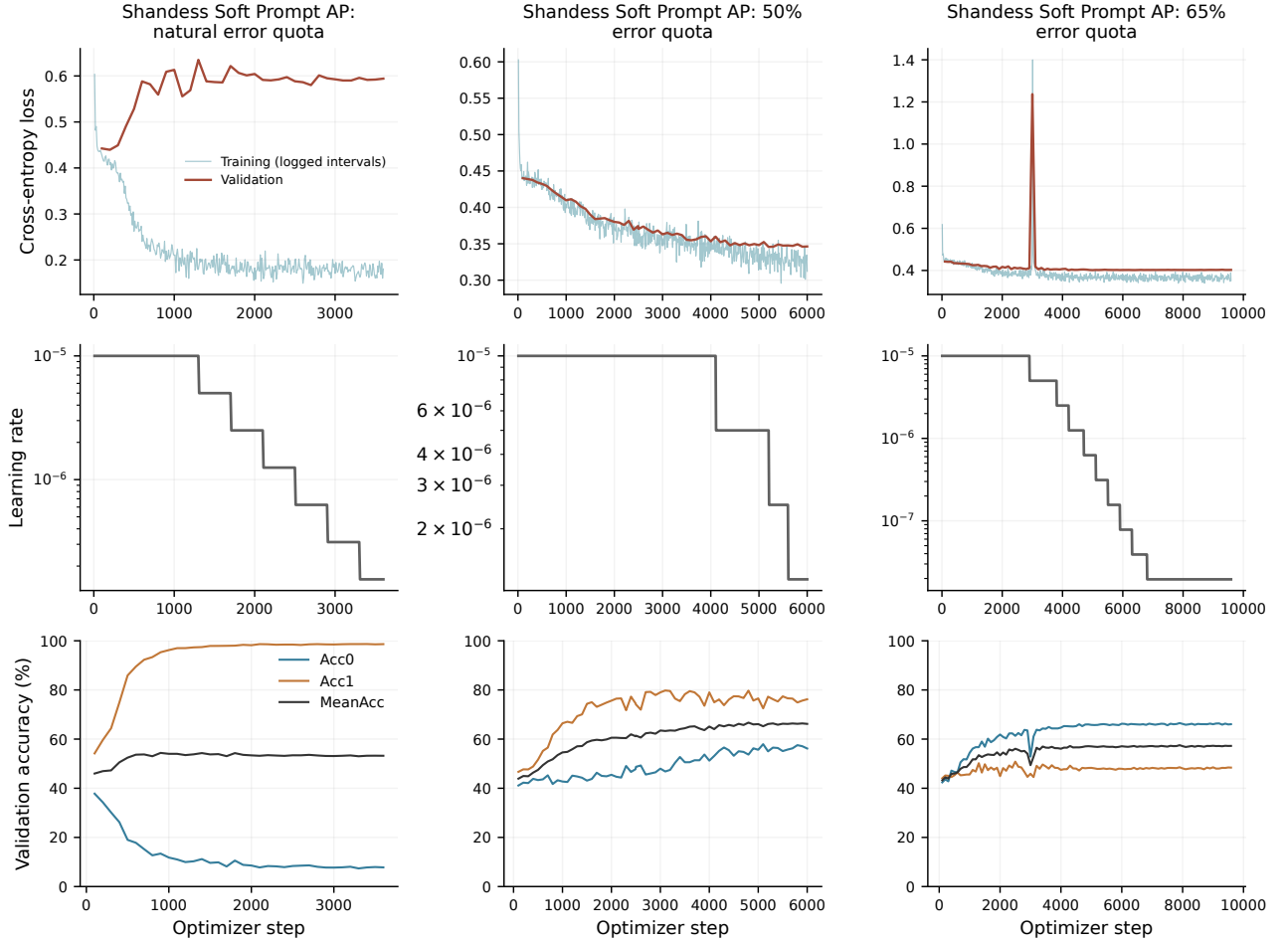


Figure 15: AP training and validation losses, learning rates and conditional validation accuracies for the natural, 50% and 65% incorrect-target quotas. Training values are logged interval losses; validation values are periodic measurements. No smoothing or interpolation of missing checkpoints is used.

Among these runs, the 50% incorrect-target quota gives the best validation MeanAcc. Raising it to 65% further improves error-response prediction, but the loss on successful responses outweighs that gain. The separate loss and conditional-accuracy curves document this trade-off through training.

A.3 Balancing Adaptiv’Math and MIA Seconde

The two Shandess Soft Prompt h1 adaptations share their architecture, initialization and validation distribution. They differ in AM/MIA target sampling: natural denotes no explicit source quota, while 50/50 imposes equal quotas. The realized natural mixture also depends on target eligibility and correctness balancing. Their window counts and selected steps differ.

AM/MIA train	AM Acc ₀	AM Acc ₁	AM mean	MIA Acc ₀	MIA Acc ₁	MIA mean	Global mean
Natural	80.22	84.56	82.39	83.64	61.57	72.61	80.29
50/50	81.77	82.88	82.33	78.54	70.18	74.36	80.19

Table 20: AM/MIA adaptation comparison: validation accuracies (%). Both start from Shandess Soft Prompt @6600, use minimum history one, two tokens, no Transformer and no metadata. Checkpoints: natural @2400; 50/50 @4000. Training window counts differ.

AM/MIA train	AM Acc ₀	AM Acc ₁	AM mean	MIA Acc ₀	MIA Acc ₁	MIA mean	Global mean
Natural	79.30	81.84	80.57	83.33	60.13	71.73	78.05
50/50	80.55	80.98	80.76	75.25	67.32	71.29	77.62

Table 21: AM/MIA adaptation comparison: test accuracies (%). Both start from Shandess Soft Prompt @6600, use minimum history one, two tokens, no Transformer and no metadata. Checkpoints: natural @2400; 50/50 @4000. Training window counts differ.

The MIA validation advantage of the 50/50 adaptation does not persist on the common test. The natural adaptation’s overall test advantage is 17 correct predictions; its paired interval includes zero (Table 35). These runs therefore do not establish a consistently superior source mixture. The original Shandess Soft Prompt h10 has a different training-history threshold and initialization, so its comparison with either adaptation does not isolate the source-ratio effect.

A.4 Initializations and KT training trajectories

Table 22 distinguishes the reused components and trainable modules of the main lineages. T and M denote the HEc Transformer and metadata additions described in Section 5.4. In the T+M lineage, phase 1 loads LoRA from Passthrough²⁹ at 4300 and HEc/projection from Shandess Soft Prompt⁵ at 8200.

Lineage / phase	Reused components	Training interpretation
Historical Shandess HiCA phase 1	Passthrough; Shandess Soft Prompt HEc and projection	Frozen input modules; train new cross-attention and gates.
Shandess HiCA + T + M, HEc frozen ²⁶ , modular phase 2	Complete Shandess HiCA + T + M, phase 1 ²⁵ @ 7300	Keep HEc frozen; adapt LoRA, projection, cross-attention and gates.
Shandess HiCA + T + M, joint ⁶ , joint phase 2	Complete Shandess HiCA + T + M, phase 1 ²⁵ @ 7300	Jointly adapt history encoder, projection, LoRA and Shandess HiCA modules.
Shandess HiCA h10 ²	HEc and projection from Shandess Soft Prompt h10 ¹ @ 6600	Distinct Shandess HiCA training; not initialized with a complete Shandess Soft Prompt or historical Shandess HiCA checkpoint.
The two Shandess Soft Prompt adaptations ^{11,12}	LoRA, HEc, projection and gate from Shandess Soft Prompt h10 ¹ @ 6600	New adaptation phase, local counter and initial LR 10^{-6} .

Table 22: Distinct checkpoint-initialization strategies. Frozen sentence encoders remain outside every online training phase.

Resumed allocations share an optimizer-step counter. A new adaptation phase has its own counter and learning-rate schedule; its compute accounting includes the reused prerequisite once. The following figures cover distinct runs: the initial models and the later h1 training/adaptations.

²⁹gamma2/PASSTHROUGH ⁵gamma42/SOFT ²⁶gamma43/FLAMINGO ²⁵gamma42/FLAMINGO ⁶gamma44/FLAMINGO ²gamma6/FLAMINGO
¹gamma6/SOFT ¹¹gamma111/SOFT ¹²gamma1111/SOFT

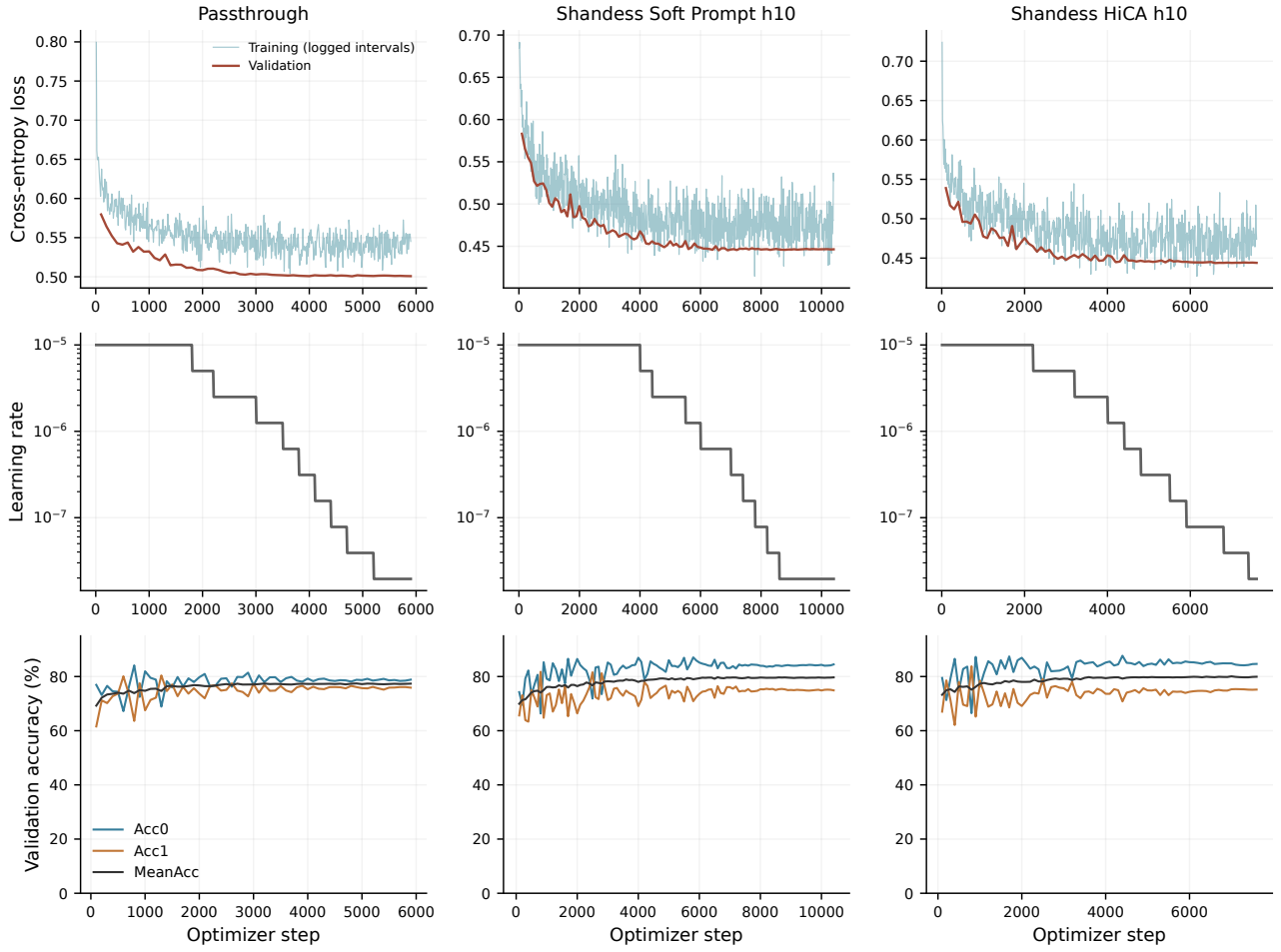


Figure 16: KT training trajectories for Passthrough, Shandess Soft Prompt h10 and Shandess HiCA h10: logged training loss and periodic validation loss, learning rate, and validation Acc_0 , Acc_1 and MeanAcc. Each column uses its run's step counter. Resumed segments are merged by step; losses are not smoothed.

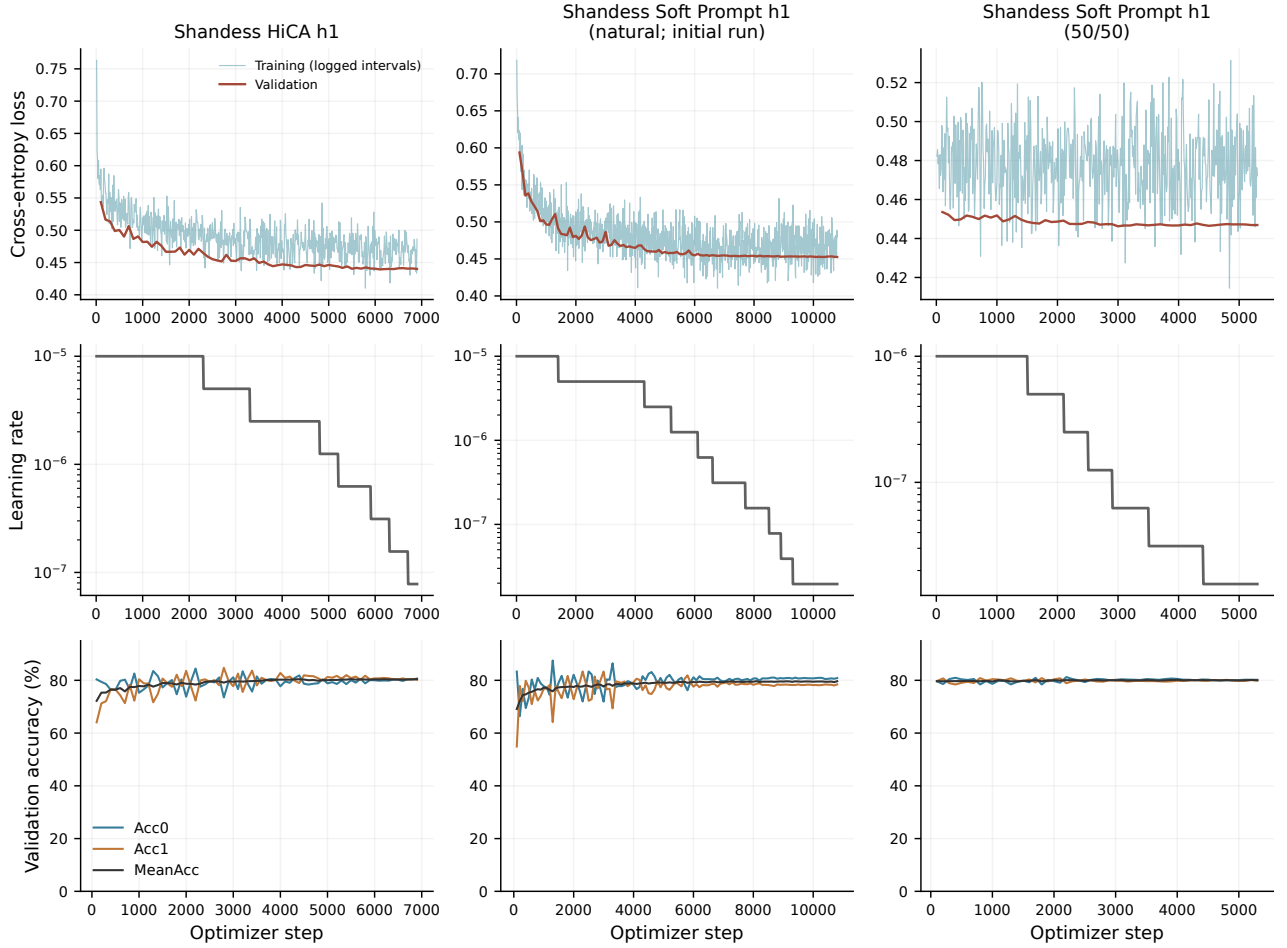


Figure 17: Additional KT training trajectories: Shandess HiCA h1, Shandess Soft Prompt with natural AM/MIA sampling, and the Shandess Soft Prompt adaptation with 50/50 AM/MIA sampling. Each column uses its own optimizer-step counter. Logged training loss, validation loss, learning rate and validation accuracies remain separate.

A.5 Computational setup and costs

The main hardware and optimization settings are given in Section 6.5. Historical one-token runs used one H100; later runs used four. The Shandess Soft Prompt h10 and Shandess HiCA h10 runs have a global batch of 80, whereas Passthrough⁸ uses 128. Older runs retain their recorded batch and accumulation settings. The joint T+M continuation uses learning-rate multipliers of 0.5 for Qwen and 1 for HEC, projection and cross-attention.

Table 23 converts observed progress-bar loop timers to H100-hours up to the selected checkpoint. It excludes setup, offline embeddings, final checkpoint-write completion and discarded or failed work. Table 24 instead sums historical allocations, including work after the best checkpoint; its full-pipeline column adds prerequisite lineages. These two accounting boundaries are not interchangeable.

⁸DDPshandess2/PASSTHROUGH

Selected checkpoint	Direct lower bound	Including Shandess Soft Prompt prerequisite
Passthrough ⁸ @ 4900	5.667	5.667
Shandess Soft Prompt h10 ¹ @ 6600	9.609	9.609
Shandess HiCA h10 ² @ 7000	7.303	16.912
Shandess Soft Prompt h1 (natural) ¹¹ @ 2400	3.507	13.116
Shandess Soft Prompt h1 (50/50) AM/MIA ¹² @ 4000	5.854	15.463

Table 23: Observed loop-time lower bounds in H100-hours up to selected checkpoints, all using four H100 GPUs. Shandess Soft Prompt h10¹ counts its initial trajectory to step 5300 and the resumed updates to 6600; the prerequisite is reused once when added to a downstream lineage.

Historical lineage	GPUs	Direct wall	Direct H100	Full H100
Normal	4	6:00:28	24:01:52	24:01:52
Passthrough	1	4:00:22	4:00:22	4:00:22
Shandess Soft Prompt, initial 1-token	1	6:00:19	6:00:19	10:00:41
Shandess Soft Prompt h10 with metadata	1 then 4	4:22:41	16:23:50	20:24:12
Shandess Soft Prompt h10 with metadata + Transformer	4	4:00:05	16:00:20	16:00:20
Shandess HiCA, initial 1-token	1	7:55:19	7:55:19	17:56:00
Shandess HiCA h10 with metadata	4	4:00:36	16:02:24	36:26:36
Shandess HiCA h10 with metadata + Transformer	4	5:30:13	22:00:52	42:01:34

Table 24: Historical training costs in hours:minutes:seconds. These are the initial 1-token models³⁶ and metadata-containing lineages, with their prerequisite stages. They do not describe the Shandess Soft Prompt h10¹, Shandess HiCA h1³, or source-mixture adaptations.

B Complete evaluation results

These tables retain the h10 checkpoints, HEc variants and AM/MIA adaptations beyond the fixed comparison in Section 7.

B.1 Knowledge Tracing

All configurations use the common 4,000-target test described in Section 7.1. Target identities, order, exercise IDs and labels in the regenerated Normal/Shandess Soft Prompt-adaptation export match the preserved predictions. The export does not establish byte-identical histories. Accuracy equals MeanAcc on this balanced test.

⁸DDPshandess2/PASSTHROUGH ¹gamma6/SOFT ²gamma6/FLAMINGO ¹¹gamma111/SOFT ¹²gamma1111/SOFT ³⁶gamma2
³gamma8/FLAMINGO

Model	Acc. = MeanAcc	Acc ₀	Acc ₁	AUC
Shandess HiCA h1 ³	78.375	80.550	76.200	0.8613
Shandess HiCA h10 ²	78.350	79.150	77.550	0.8588
Shandess HiCA + T + M ⁶	78.250	79.600	76.900	0.8579
Shandess Soft Prompt h1 (natural) ¹¹	78.050	80.900	75.200	0.8574
Shandess Soft Prompt + T + M ⁵	77.900	78.750	77.050	0.8561
Shandess Soft Prompt h1 (50/50) ¹²	77.625	78.450	76.800	0.8560
Shandess Soft Prompt h1 (natural; initial run) ¹⁰	77.275	78.900	75.650	0.8543
Shandess Soft Prompt h10 ¹	77.225	77.550	76.900	0.8499
Shandess Soft Prompt + T ⁴	76.825	78.000	75.650	0.8474
DKT	75.975	74.500	77.450	0.8340
Shandess Soft Prompt, 1 token + T + M ⁷	75.950	74.500	77.400	0.8374
Normal ²⁸	75.875	77.000	74.750	0.8335
LKT	75.750	77.000	74.500	0.8342
Passthrough ⁸	75.350	77.500	73.200	0.8194
qDKT-ID	74.200	73.450	74.950	0.8161
qDKT-text	74.100	73.200	75.000	0.8177

Table 25: Common KT test: 4,000 aligned targets, 2,000 per class. Accuracy columns are percentages; AUC is on the unit scale. T denotes the HEc Transformer and M metadata. HEc variants use two tokens per interaction except the labelled one-token variant. Normal uses textual history.

The supplementary HEc configurations do not yield a systematic test gain: the metadata effect differs between validation and test, and the Shandess HiCA T+M result is close to the h10 result without either addition. These scores are not used to reselect the architectures. Paired intervals for the available checkpoint comparisons are given in Table 35.

Source-specific results. Global balance does not imply balance within AM or MIA; source accuracy and source MeanAcc are therefore distinct quantities.

Model	AM Acc ₀	AM Acc ₁	AM mean	MIA Acc ₀	MIA Acc ₁	MIA mean
Shandess HiCA h1 ³	82.28	79.83	81.06	77.90	67.97	72.94
Shandess HiCA h10 ²	80.88	81.84	81.36	76.52	67.81	72.16
Shandess HiCA + T + M ⁶	81.46	81.05	81.25	76.77	67.48	72.13
Shandess Soft Prompt h1 (natural) ¹¹	79.30	81.84	80.57	83.33	60.13	71.73
Shandess Soft Prompt + T + M ⁵	80.05	81.20	80.62	76.77	67.65	72.21
Shandess Soft Prompt h1 (50/50) ¹²	80.55	80.98	80.76	75.25	67.32	71.29
Shandess Soft Prompt h10 ¹	79.55	80.62	80.09	74.49	68.46	71.48
Shandess Soft Prompt + T ⁴	80.05	80.04	80.05	74.87	65.69	70.28
DKT	77.07	81.41	79.24	70.58	68.46	69.52
Shandess Soft Prompt, 1 token + T + M ⁷	76.66	82.13	79.39	71.21	66.67	68.94
Normal ²⁸	78.89	79.76	79.32	74.12	63.40	68.76
LKT	79.47	78.03	78.75	73.23	66.50	69.87
Passthrough ⁸	77.32	80.40	78.86	77.78	56.86	67.32
qDKT-ID	76.82	79.54	78.18	68.31	64.54	66.43
qDKT-text	76.82	80.33	78.58	67.68	62.91	65.29

Table 26: Conditional KT accuracies (%). AM has 1,208 failures and 1,388 successes; MIA has 792 failures and 612 successes. MeanAcc averages conditional accuracies within each source.

Published MIAAM results. The following reference table uses the paper’s protocols[2]: natural outcome distributions, histories up to 200 for classical KT and up to 100 for the text foundation-model benchmark. Its approximate MeanAcc is derived from the rounded published conditional accuracies. This balances the metric, but does not match targets, source mixture or input information to the local test. Section 7.1 discusses the imbalance between failure and success prediction.

³gamma8/FLAMINGO ²gamma6/FLAMINGO ⁶gamma44/FLAMINGO ¹¹gamma111/SOFT ⁵gamma42/SOFT ¹²gamma1111/SOFT
¹⁰gamma11/SOFT ¹gamma6/SOFT ⁴gamma7/SOFT ⁷gamma5/SOFT ²⁸gamma41/NORMAL ⁸DDPshandess2/PASSTHROUGH

Published model	Information	AUC	Acc.	Acc ₀	Acc ₁	MeanAcc $\simeq$
qDKT-ID	IDs	0.85	0.89	0.55	0.94	0.745
qDKT-text	Text embeddings	0.85	0.89	0.56	0.93	0.745
DKT	Activity IDs	0.84	0.87	0.50	0.94	0.720
GKT	Activity graph	0.84	0.86	0.47	0.94	0.705
Mistral Small 4	Text	0.83	0.82	0.33	0.97	0.650
Gemma-4-31B	Text	0.88	0.85	0.62	0.92	0.770
GPT-5.5	Text	0.89	0.85	0.54	0.94	0.740
Mistral Small 4	Text + PD	0.82	0.82	0.29	0.97	0.630
Gemma-4-31B	Text + PD	0.88	0.86	0.65	0.91	0.780
GPT-5.5	Text + PD	0.88	0.85	0.56	0.94	0.750

Table 27: Reported MIAAM KT results, Table 3 of the supplied dataset manuscript [2]; all values on the unit scale. PD means pedagogical descriptions. MeanAcc is derived from rounded conditional accuracies. These rows do not share the common-test protocol.

B.2 Answer Prediction

The recent AP test contains 4,000 targets: 357 incorrect and 3,643 correct responses, split between 2,900 AM and 1,100 MIA targets. Uniform random choice among the supplied options has expected accuracy 46.373%. Conditional accuracies distinguish prediction of student errors from solving the exercise.

Model / checkpoint	Accuracy	Acc ₀	Acc ₁	MeanAcc
Always the correct answer	91.075	0.000	100.000	50.000
Shandess Soft Prompt AP, 50% errors ¹³ @ 2400	76.450	44.818	79.550	62.184
Shandess Soft Prompt AP, 65% errors ¹⁵ @ 7900	50.525	62.465	49.355	55.910

Table 28: Recent AP test results (%). Both learned models use a two-token HEc without Transformer or metadata, with AM/MIA training targets balanced 50/50. Shandess Soft Prompt AP, 50% errors¹³ was tested before its later validation improvement; Shandess Soft Prompt AP, 65% errors¹⁵ reloads its best validation checkpoint after completing training.

The 50% run was tested at step 2400, before its best recovered validation checkpoint at 4800. The 65% run was tested at its best validation checkpoint, 7900. Thus these tests describe the available trained systems, with different budgets; they neither test the later 50% checkpoint nor compare all AP architectures under a matched protocol.

Source and answer-set size. The option count is supplied by the window builder from exercise content and observed answer indices. All AM targets have two options; larger answer sets belong to MIA. Table 29 gives source and option-count marginals, while Table 30 crosses the two factors. The MIA two-option counts are obtained by subtracting AM counts from the global two-option group, separately for correct and incorrect responses. Pooled five-to-eight-option scores are also aggregated by counts.

¹³gamma91/SOFT ¹⁵gamma94/SOFT

Group	Run	Targets	Accuracy	Acc ₀	Acc ₁	MeanAcc
AM	Shandess Soft Prompt AP, 50% errors ¹³	2900	80.00	53.19	81.37	67.28
AM	Shandess Soft Prompt AP, 65% errors ¹⁵	2900	48.10	75.89	46.68	61.29
MIA	Shandess Soft Prompt AP, 50% errors ¹³	1100	67.09	39.35	73.87	56.61
MIA	Shandess Soft Prompt AP, 65% errors ¹⁵	1100	56.91	53.70	57.69	55.70
2 options	Shandess Soft Prompt AP, 50% errors ¹³	3427	80.54	50.53	82.30	66.41
2 options	Shandess Soft Prompt AP, 65% errors ¹⁵	3427	52.35	72.63	51.16	61.90
3 options	Shandess Soft Prompt AP, 50% errors ¹³	191	51.83	47.54	53.85	50.69
3 options	Shandess Soft Prompt AP, 65% errors ¹⁵	191	41.36	59.02	33.08	46.05
4 options	Shandess Soft Prompt AP, 50% errors ¹³	179	52.51	36.36	59.68	48.02
4 options	Shandess Soft Prompt AP, 65% errors ¹⁵	179	39.11	49.09	34.68	41.88
5+ options	Shandess Soft Prompt AP, 50% errors ¹³	203	51.72	29.41	59.21	44.31
5+ options	Shandess Soft Prompt AP, 65% errors ¹⁵	203	38.42	43.14	36.84	39.99

Table 29: Recent AP subgroup results (%). Bold compares the two models within each group. Source groups and option-count groups are separate marginal breakdowns, not independent datasets. The 5+ group pools 203 targets with five to eight options using counts before computing conditional means.

Model	Source	Options	n_0	n_1	Acc ₀	Acc ₁	MeanAcc
Shandess Soft Prompt 50% ¹³	AM	2	141	2759	53.19	81.37	67.28
Shandess Soft Prompt 50%	MIA	2	49	478	42.86	87.66	65.26
Shandess Soft Prompt 50%	MIA	3	61	130	47.54	53.85	50.69
Shandess Soft Prompt 50%	MIA	4	55	124	36.36	59.68	48.02
Shandess Soft Prompt 50%	MIA	5+	51	152	29.41	59.21	44.31
Shandess Soft Prompt 65% ¹⁵	AM	2	141	2759	75.89	46.68	61.29
Shandess Soft Prompt 65%	MIA	2	49	478	63.27	76.99	70.13
Shandess Soft Prompt 65%	MIA	3	61	130	59.02	33.08	46.05
Shandess Soft Prompt 65%	MIA	4	55	124	49.09	34.68	41.88
Shandess Soft Prompt 65%	MIA	5+	51	152	43.14	36.84	39.99

Table 30: AP test accuracies (%) by source and option count. The models use 50% or 65% incorrect training targets, respectively, at checkpoints 2400 and 7900. Counts n_0, n_1 refer to incorrect/correct student responses. MIA with two options is reconstructed by subtracting AM counts from the two-option totals; options 5–8 are pooled by counts.

Matching the answer-set size reduces the AM/MIA gap for the 50% model, indicating that larger answer sets contribute to the aggregate source gap. It does not match exercise difficulty. The training-quota trade-off also depends on the stratum: 65% improves MeanAcc on two-option MIA but reduces it on AM. The five-to-eight-option group has only 51 incorrect targets, limiting conclusions about distractor prediction there. The logged metric `exercice_accuracy{0}`, which measures solving the exercise, is not Acc₀.

Published AP reference. The MIAAM AP table uses a separate evaluation protocol and reports global and option-count accuracies, without the conditional Acc₀/Acc₁ breakdown used above. Always returning the correct answer reaches 92% under that protocol.

¹³gamma91/SOFT ¹⁵gamma94/SOFT

Published reference	Accuracy	2 options	3 options	4 options
Random option	0.48	0.50	0.33	0.25
Correct answer	0.92	0.96	0.80	0.80
Mistral Small 4	0.56	0.58	0.10	0.30
Gemma-4-31B	0.90	0.90	0.90	0.73
GPT-5.5	0.88	0.89	0.90	0.69

Table 31: Reported AP accuracies from Table 4 of the supplied MIAAM manuscript [2], on the unit scale. The table does not report foundation-model accuracy conditional on student correctness.

B.3 Probability quality

The full Brier/ECE table extends Section 7.3 to all checkpoints with archived probabilities. ECE uses ten fixed equal-width probability bins B_m , shared by all models:

$$\text{ECE}_{10} = \sum_m \frac{|B_m|}{n} \left| \frac{1}{|B_m|} \sum_{i \in B_m} p_i - \frac{1}{|B_m|} \sum_{i \in B_m} c_i \right|. \quad (13)$$

Empty bins contribute zero; no post-hoc calibration is fitted on the test. All reliability curves and bin counts remain in the repository.

Model	Brier	ECE ₁₀
Shandess HiCA h1 ³	0.1509	0.0128
Shandess HiCA h10 ²	0.1522	0.0154
Shandess HiCA + T + M ⁶	0.1526	0.0138
Shandess Soft Prompt h1 (natural) ¹¹	0.1532	0.0132
Shandess Soft Prompt + T + M ⁵	0.1541	0.0154
Shandess Soft Prompt h1 (50/50) ¹²	0.1541	0.0149
Shandess Soft Prompt h10 ¹	0.1575	0.0265
Shandess Soft Prompt + T ⁴	0.1583	0.0202
DKT	0.1656	0.0186
Shandess Soft Prompt, 1 token + T + M ⁷	0.1648	0.0346
Normal ²⁸	0.1662	0.0196
LKT	0.1690	0.0550
Passthrough ⁸	0.1729	0.0209
qDKT-ID	0.1765	0.0361
qDKT-text	0.1751	0.0315

Table 32: Probability metrics on the common balanced KT test. Lower is better. ECE uses ten fixed equal-width bins; no calibration model is fitted.

Both Shandess Soft Prompt h1 adaptations improve Brier and ECE over the h10 checkpoint. Brier and ECE need not rank models identically: Passthrough has lower ECE than Shandess Soft Prompt h10 despite its worse Brier score. These measurements apply to the balanced test, not the natural deployment distribution.

B.4 Latency and model constraints

The full sweep supplements the batch-one timing protocol in Section 7.2. The baseline benchmark uses an H100 80 GB GPU, FP32 for recurrent models and BF16 autocast for LKT. Shandess checkpoints were measured in separate jobs with their recorded inference configuration. The common timing boundary includes HEc but excludes input preparation and transfers; differences between jobs are not effects of the training mixture.

³gamma8/FLAMINGO ²gamma6/FLAMINGO ⁶gamma44/FLAMINGO ¹¹gamma111/SOFT ⁵gamma42/SOFT ¹²gamma1111/SOFT
¹gamma6/SOFT ⁴gamma7/SOFT ⁷gamma5/SOFT ²⁸gamma41/NORMAL ⁸DDPshandess2/PASSTHROUGH

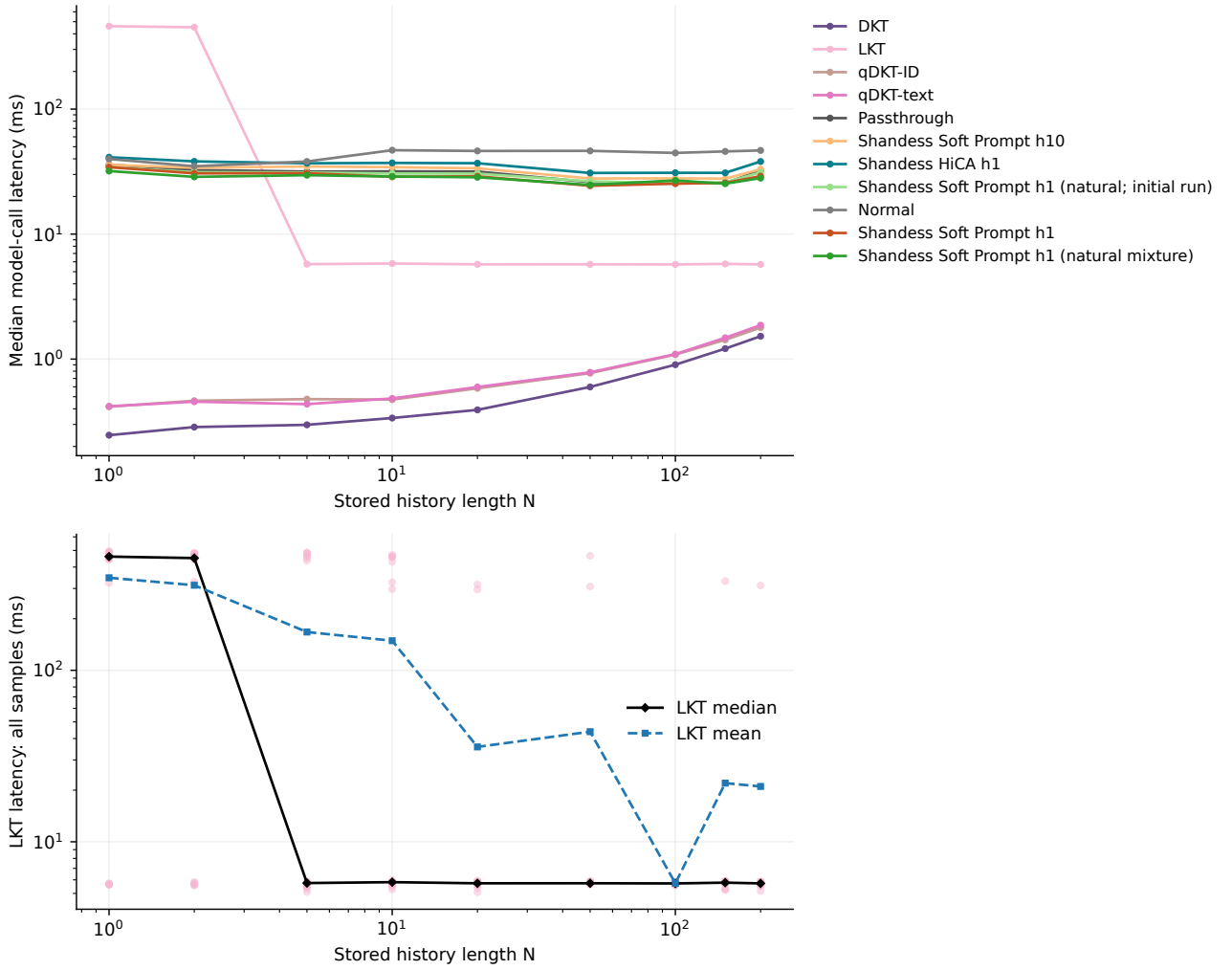


Figure 18: Median KT model-call latency by stored history length, batch one. The recurrent baselines are much faster than the Qwen-based models. LKT has substantial variability; its discontinuity is not evidence that a longer history makes it causally faster. Timings exclude input preparation.

The LKT mean and median diverge because some calls are much slower; the curve is not evidence that longer histories cause lower latency. For Shandess, HEc padding to the configured maximum and different target texts across length groups affect the workload. Some groups cycle over fewer than twenty distinct examples. Recurrent baselines replay the supplied history rather than reuse an incremental state.

Configuration	Total predictor parameters	Median latency (ms)
DKT	0.540 M	1.524
qDKT-ID	7.147 M	1.785
qDKT-text	9.404 M	1.868
LKT	$\simeq 110$ M	5.720
Normal	$\simeq 502.8$ M	46.663
Passthrough ⁸	502.8 M	32.282
Shandess Soft Prompt h10 ¹	515.1 M	33.013
Shandess Soft Prompt h1 (50/50) ¹²	515.1 M	29.078
Shandess Soft Prompt h1 (natural) ¹¹	515.1 M	28.035
Shandess HiCA h1 ³	$\simeq 572.9$ M	38.070

Table 33: Local predictor size and median model-call latency at stored history length 200, batch size one. LKT’s mean latency is 21.022 ms at this length. DKT and qDKT counts are calculated from the implemented architectures and vocabulary sizes, rounded to three decimals in millions. Offline semantic encoders are excluded from predictor parameter counts. Normal uses the same Qwen/LoRA size as Passthrough; the Shandess HiCA h1 uses the same predictor configuration as the Shandess HiCA h10. Adaptation changes weights without adding parameters.

Model family	New students	New exercise and history constraints
DKT	Shared population model; observed short-history results	Requires an activity mapping; compact recurrent history
qDKT-ID	Shared population model; observed short-history results	Fixed exercise vocabulary; exercise-ID output mapping
qDKT-text	Shared population model; observed short-history results	Semantic initialization; exercise-specific output mapping
LKT	Student-disjoint test; observed short-history results	Text input; 512-token budget strongly truncates history
Passthrough	Population prior; no personalization through history	Current-exercise text; no historical context
Shandess Soft Prompt / Shandess HiCA	Student-disjoint test; fixed-target history sweep	Initial AM test-only evaluation; compact history tokens

Table 34: Architectural suitability and measured evidence. Accepting a new exercise representation is distinct from demonstrating accurate prediction for exercises absent from training.

C Uncertainty of the KT accuracy differences

Predictions from the common test are paired by window. The 4,000 windows represent 2,363 students. To estimate uncertainty in model differences, we resample students with replacement, keeping their windows together and using the same sampled students for both models. The table reports percentile intervals from 2,000 replicates (seed 20260908). The measured quantity is the difference in accuracy; class proportions can vary between replicates.

⁸DDPshandess2/PASSTHROUGH ¹gamma6/SOFT ¹²gamma1111/SOFT ¹¹gamma111/SOFT ³gamma8/FLAMINGO

Paired comparison	Difference	95% interval
Shandess HiCA h10 ² – Shandess Soft Prompt h10 ¹	+1.125	[+0.313, +1.964]
Shandess HiCA h10 ² – Passthrough ⁸	+3.000	[+1.836, +4.218]
Shandess HiCA h10 ² – DKT	+2.375	[+1.173, +3.558]
Shandess HiCA h1 ³ – Shandess HiCA h10 ²	+0.025	[-0.668, +0.707]
Shandess HiCA h10 ² – Shandess Soft Prompt h1 (natural) ¹¹	+0.300	[-0.519, +1.078]
Shandess HiCA h10 ² – Shandess Soft Prompt h1 (50/50) ¹²	+0.725	[-0.025, +1.520]
Shandess Soft Prompt h1 (natural) ¹¹ – Shandess Soft Prompt h1 (50/50) ¹²	+0.425	[-0.222, +1.055]
Shandess Soft Prompt h1 (50/50) ¹² – Shandess Soft Prompt h10 ¹	+0.400	[-0.200, +0.981]

Table 35: Accuracy differences in percentage points; 2,000 paired student-cluster bootstrap replicates, seed 20260908. The 4,000 windows represent 2,363 students. Exploratory intervals condition on fixed checkpoints and exclude training-seed variability.

These intervals describe variability from the evaluated student sample for fixed trained checkpoints. They do not measure variability across independent training seeds. In the short-history group, Shandess HiCA h1 gains nine predictions over the Shandess HiCA h10 (3.38 points); the corresponding interval is $[-0.75, +7.58]$ points over 266 windows and 264 students.

D Validation runs and checkpoints

This appendix preserves the recovered validation results beyond the subset needed for the architectural argument in Section 5.4 and the training comparisons in Section 6. The values follow the consolidated experiment synthesis updated with the JSON export of 10 September 2026. “Best” means the highest MeanAcc among the received complete validation records; it does not assume the current state of checkpoints on the cluster. Acc₀, Acc₁ and AUC refer to that same point when available. All scores in these tables use the unit scale and are validation results, not the common-test scores of Section 7.

A directory name alone is not a complete experimental identity: cache, initialization, task and training phase also matter. Resumed allocations form one checkpoint lineage rather than additional independent training seeds.

D.1 Validation cache identities

The main KT validations contain 10,000 windows balanced between correct and incorrect targets. Table 36 gives their cache identities to make the following tables self-contained. Different hashes are provenance information, not sufficient grounds for declaring two experiments incomparable. In particular, Shandess Soft Prompt h10¹ and Shandess Soft Prompt h1 (natural; initial run)¹⁰ change minimum history from ten to one while retaining the principal validation settings.

Cache	Suffix	Maximum N	Recovered uses
A	29d746d84bba573f	200	Main two-token variants and variants with metadata or a HEC Transformer.
B	c794e0b1ecfc8899	200	Passthrough/DoRA and source-mixture experiments.
C	cbdf23daf2504c24	200	Initial and alternative HEC lineages.
D	3d150f8cdf3395cc	200	Earlier Passthrough.
E	677925dcd07220c2	40	Normal, selected step 2900.

Table 36: Exact validation-cache suffixes recovered from the run commands. N denotes preceding interactions, excluding the prediction target.

²gamma6/FLAMINGO ¹gamma6/SOFT ⁸DDPshandess2/PASSTHROUGH ³gamma8/FLAMINGO ¹¹gamma111/SOFT ¹²gamma1111/SOFT
¹⁰gamma11/SOFT

D.2 Main KT configurations and intermediate phases

Table 37 lists scientific configurations and intermediate training phases, including variants that did not improve validation performance. T and M denote the HEc Transformer and metadata; initialization paths are given in Table 22. Accidental, initialization-issue and short-trial records remain in the repository supplement `supplementary/validation_trials.tex`.

Table 37: KT configurations and intermediate phases. All values come from the selected validation point.

Configuration	Best step	MeanAcc	Acc ₀	Acc ₁	AUC	Cache
Shandess HiCA h10 ²	7000	0.8008	0.8486	0.7530	0.8738	A
Shandess Soft Prompt h10 ¹	6600	0.7974	0.8302	0.7646	0.8729	A
Shandess Soft Prompt + T ⁴	9500	0.7968	0.8504	0.7432	0.8717	A
Shandess HiCA + T, no metadata ⁴⁹	6200	0.7956	0.8474	0.7438	0.8713	—
Shandess HiCA + T + M, joint ⁶	6600	0.7958	0.8458	0.7458	0.8729	A
Shandess Soft Prompt + T + M ⁵	8200	0.7940	0.8362	0.7518	0.8700	A
Shandess HiCA + M, phase 2 ²³	4900	0.7936	0.8394	0.7478	0.8661	A
Shandess HiCA + M, LoRA + gates ²⁴	5500	0.7927	0.8428	0.7426	0.8648	A
Shandess HiCA + T + M, HEc frozen ²⁶	4800	0.7926	0.8434	0.7418	0.8683	A
Shandess HiCA + T + M, phase 1 ²⁵	7300	0.7916	0.8352	0.7480	0.8680	A
Shandess HiCA + M, extended phase 1 ²²	7100	0.7899	0.8348	0.7450	0.8644	A
Shandess HiCA + M, phase 1 ²¹	3100	0.7885	0.8278	0.7492	0.8623	A
Shandess HiCA, initial 1 token ¹⁹	3600	0.7880	0.8362	0.7398	0.8620	C
Shandess HiCA, alternative HEc ³²	1700	0.7878	0.8170	0.7586	0.8617	C
Shandess Soft Prompt + M ²⁰	3700	0.7864	0.8454	0.7274	0.8635	A
Shandess Soft Prompt, initial 1 token ¹⁸	4300	0.7802	0.8230	0.7374	0.8578	C
Shandess Soft Prompt, 1 token + T + M ⁷	4900	0.7763	0.8184	0.7342	0.8510	A
Passthrough ⁸	4900	0.7745	0.7880	0.7610	0.8363	B
Shandess Soft Prompt, alternative HEc ³¹	1200	0.7730	0.8022	0.7438	0.8448	C
Passthrough, earlier run ²⁹	4400	0.7712	0.7962	0.7462	0.8330	D
Normal, text history ²⁸	2900	0.7694	0.8042	0.7346	0.8412	E
Passthrough DoRA ⁹	5800	0.7683	0.7912	0.7454	0.8296	B

The Shandess HiCA h10² row is the no-Transformer run 1622421_1; an earlier failed attempt does not define its architecture.

²gamma6/FLAMINGO ¹gamma6/SOFT ⁴gamma7/SOFT ⁴⁹ ⁶gamma44/FLAMINGO ⁵gamma42/SOFT ²³gamma4/FLAMINGO
²⁴gamma41/FLAMINGO ²⁶gamma43/FLAMINGO ²⁵gamma42/FLAMINGO ²²gamma35/FLAMINGO ²¹gamma3/FLAMINGO
¹⁹gamma2/FLAMINGO ³²nhec/FLAMINGO ²⁰gamma3/SOFT ¹⁸gamma2/SOFT ⁷gamma5/SOFT ⁸DDPshandess2/PASSTHROUGH
³¹nhec/SOFT ²⁹gamma2/PASSTHROUGH ²⁸gamma41/NORMAL ⁹DDPshandess/PASSTHROUGH

D.3 Shandess Soft Prompt h1 runs

The following runs use two HEC tokens, no metadata or Transformer, minimum training history one, and balanced correctness targets. Adaptations^{11,12} load the complete Shandess Soft Prompt h10 checkpoint at 6600 and use local step counters. They are distinct from the initial h1 run¹⁰. Their source-mixture comparison is discussed in Appendix A.3.

Table 38: Later KT Shandess Soft Prompt validation runs, all on cache B. Complete-initialization adaptations are Shandess Soft Prompt h1 (natural)¹¹ and Shandess Soft Prompt h1 (50/50) AM/MIA¹²; scores are not attributed to the distinct Shandess Soft Prompt h1 (natural; initial run)¹⁰ test.

Configuration	Best step	MeanAcc	Acc ₀	Acc ₁	AUC	Cache
Shandess Soft Prompt h1 (natural; initial run) ¹⁰	8100	0.7969	0.8074	0.7864	0.8696	B
Shandess Soft Prompt h1 (natural) ¹¹	2400	0.8029	0.8152	0.7906	0.8763	B
Shandess Soft Prompt h1 (50/50) AM/MIA ¹²	4000	0.8019	0.8054	0.7984	0.8737	B

D.4 Shandess HiCA h1 checkpoint record

Configuration	Best step	MeanAcc	Acc ₀	Acc ₁
Shandess HiCA h1 ³	5500	0.8047	0.8008	0.8086

Table 39: Additional validation entries. For Shandess HiCA h1, AUC at step 5500 is 0.8776. The existing tests use step 6900, whose validation MeanAcc is 0.8044 (Acc₀ 0.8066, Acc₁ 0.8022); those test scores are not attributed to step 5500.

D.5 Other historical protocols

Table 40 preserves all twelve remaining historical rows in the consolidated inventory, with their original KT or AP protocol. These form a separate historical table. AP does not use binary KT AUC in this inventory.

Table 40: Other historical validation results under their historical protocols.

Configuration	Best step	MeanAcc	Acc ₀	Acc ₁	AUC	Cache
Shandess HiCA KT, historical 1 ³⁸	1340	0.7609	0.7789	0.7429	0.8336	—
Passthrough, early trial ³³	1800	0.7589	0.7582	0.7596	0.8212	—
Shandess HiCA KT, historical 2 ³⁹	2680	0.7568	0.7944	0.7193	0.8322	—
Shandess HiCA KT, historical 3 ⁴⁰	1206	0.7502	0.7750	0.7255	0.8213	—
Shandess Soft Prompt KT, historical 3-3 ⁴⁵	1876	0.7388	0.7353	0.7422	0.8053	—
Shandess HiCA AP, historical 1 ⁴²	1755	0.7387	0.7481	0.7293	—	—
Shandess Soft Prompt KT, historical 3-2 ⁴⁵	2025	0.7270	0.7957	0.6583	0.7954	—

Continued on the next page

¹¹gamma111/SOFT ¹²gamma1111/SOFT ¹⁰gamma11/SOFT ³gamma8/FLAMINGO ³⁸kt-qwen-hec-FLAMINGO-LORA-1
³³shandess/PASSTROUGH ³⁹kt-qwen-hec-FLAMINGO-3 ⁴⁰kt-qwen-hec-FLAMINGO-4 ⁴⁵kt-qwen-hec-SOFT
⁴²fs-qwen-hec-FLAMINGO-55704

Continued from the previous page

Configuration	Best step	MeanAcc	Acc ₀	Acc ₁	AUC	Cache
Passthrough KT, historical 1 ⁴⁴	1152	0.7044	0.8164	0.5925	0.7692	—
Shandess Soft Prompt KT, historical 3 ⁴⁵	1890	0.5210	0.4872	0.5548	0.5500	—
Passthrough AP, historical 1 ⁴⁶	1071	0.5101	0.4891	0.5310	—	—
Shandess HiCA AP, historical 2 ⁴⁷	138	0.2245	0.4490	0.0000	—	—
Shandess HiCA AP, historical 3 ⁴⁸	276	0.2245	0.4490	0.0000	—	—

D.6 AP configurations

Table 19 reports the natural, 50% and 65% incorrect-target runs; their trajectories appear in Figure 15. The mixed-distribution AP continuation¹⁶ reaches validation MeanAcc 0.5521 at step 3800. Its earlier phase used natural sampling before switching to 65% incorrect targets, so it is excluded from the fixed-quota comparison.

E Cold-start KT details and uncertainty

The completed student and exercise prediction exports are documented in `data/cold_start/`. Six fixed checkpoints are evaluated: Passthrough⁸ at 4900; Shandess Soft Prompt h10¹ at 6600; Shandess Soft Prompt h1 with AM/MIA 50/50¹² at 4000 and natural sampling¹¹ at 2400; Shandess HiCA h10² at 7000; and Shandess HiCA h1³ at 6900. No weights change during evaluation.

E.1 Naturally available histories

The common-test windows are grouped by stored history length N . Models share targets within a bin, but target populations differ across bins. The $N = 200$ group is separate because the cap concentrates 1,403 windows there. Passthrough can vary across bins despite ignoring history, since current exercises and students differ. No zero-history condition was evaluated.

⁴⁴kt-qwen-hec-PASSTHROUGH ⁴⁵kt-qwen-hec-SOFT ⁴⁶fs-qwen-hec-PASSTHROUGH-56716 ⁴⁷fs-qwen-hec-FLAMINGO-2185494
⁴⁸fs-qwen-hec-FLAMINGO-2186547 ¹⁶gamma9/SOFT ⁸DDPshandess2/PASSTHROUGH ¹gamma6/SOFT ¹²gamma1111/SOFT
¹¹gamma111/SOFT ²gamma6/FLAMINGO ³gamma8/FLAMINGO

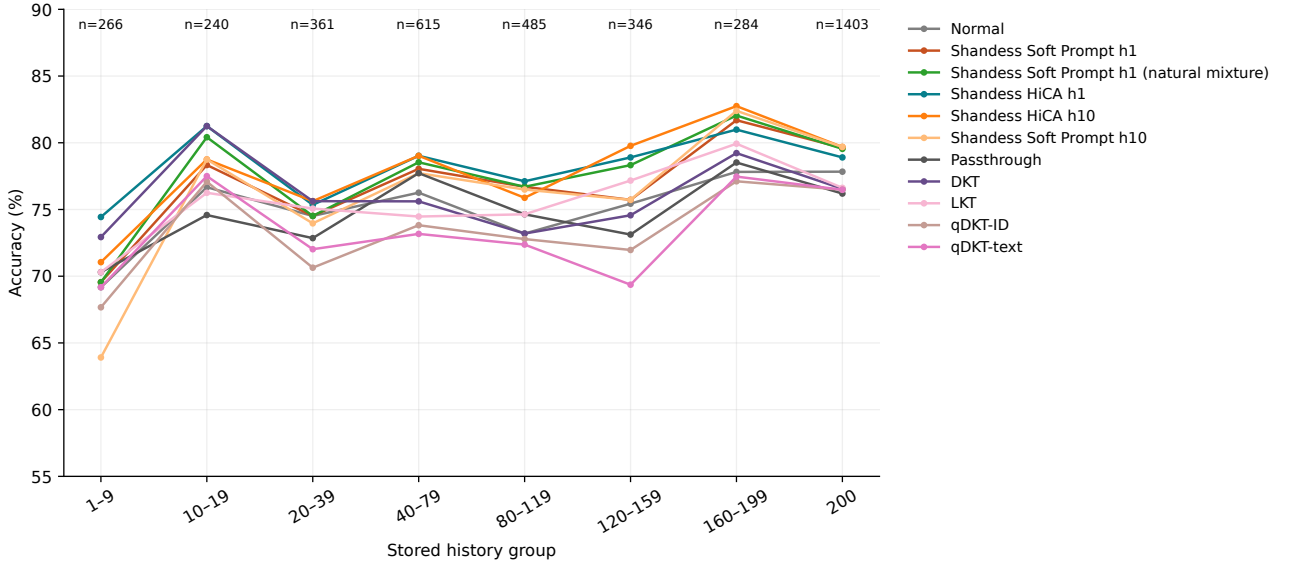


Figure 19: KT performance grouped by naturally available, stored history length. The selected checkpoints and baselines share the same windows in each bin. Variation across bins combines differences in available history and in target population; it is not a truncation experiment at fixed targets.

Model	Accuracy, $N = 1-9$ (%)
Shandess HiCA h1 ³	74.44
DKT	72.93
Shandess HiCA h10 ²	71.05
LKT	70.30
Passthrough ⁸	70.30
Shandess Soft Prompt h1 (50/50) ¹²	69.55
Shandess Soft Prompt h1 (natural) ¹¹	69.55
Normal ²⁸	69.17
qDKT-text	69.17
qDKT-ID	67.67
Shandess Soft Prompt h10 ¹	63.91

Table 41: Accuracy on the same 266 targets with one to nine preceding interactions. Exact lengths are regrouped consistently; these conditional accuracies are not necessarily MeanAcc.

On the 266 targets with one to nine preceding interactions, the balanced Shandess Soft Prompt h1 adaptation improves over h10 by 15 correct predictions (5.64 points; exploratory paired interval [0.38, 10.86]). The corresponding Shandess HiCA h1–h10 interval is given in Appendix C. These subgroup estimates use 264 students and differ from the fixed-target experiment below. Stored length is also not effective input length for every model: LKT truncates 3,906 of 4,000 windows (97.65%), retaining on average 2.979 interactions within its 512-token budget.

E.2 Fixed-target history sweep

Each target has at least 200 preceding interactions. Every checkpoint receives the most recent $N \in \{1, 2, 5, 10, 20, 50, 100, 150, 200\}$ interactions, in chronological order, while student, exercise, response and target identity remain fixed. All 54 checkpoint–length evaluations have complete, aligned predictions. Passthrough predictions are identical at every length.

³gamma8/FLAMINGO ²gamma6/FLAMINGO ⁸DDPshandess2/PASSTHROUGH ¹²gamma1111/SOFT ¹¹gamma111/SOFT ²⁸gamma41/NORMAL
¹gamma6/SOFT

KT cohort	Targets	Students	AM / MIA	n_0	n_1
Fixed targets	4,000	1,075	3,507 / 493	1,004	2,996
Test-only exercises	66	21	66 / 0	14	52
Seen-exercise controls	4,000	1,842	4,000 / 0	718	3,282

Table 42: Cold-start evaluation cohorts. Each checkpoint receives the same targets within each experiment. n_0 and n_1 count observed failures and successes; student counts are not additive across cohorts.

None of the cohorts in Table 42 imposes a correctness quota, so accuracy and MeanAcc differ. The fixed-target median position in the student trajectory is 385. Truncation therefore measures accessible information for those targets, not performance on actual first attempts.

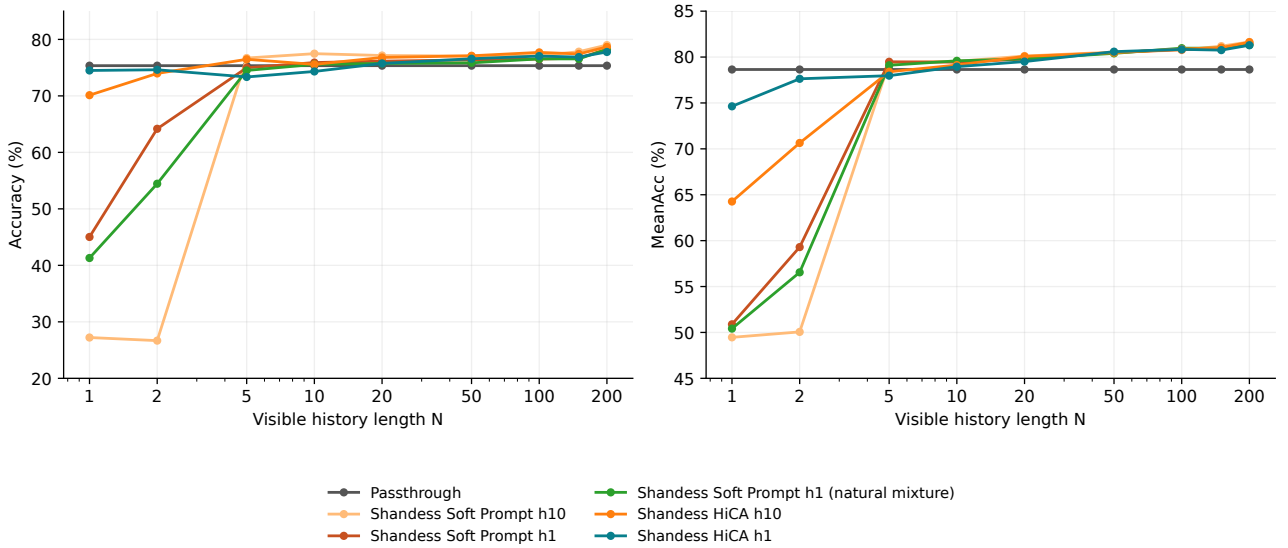


Figure 20: KT performance on fixed targets, with all nine observed lengths and a logarithmic horizontal axis. h1/h10 denote the minimum training history; the Shandess Soft Prompt h1 models are adaptations of the Shandess Soft Prompt h10 checkpoint. Balanced and natural identify their AM/MIA training mixtures. Points are descriptive, not averages across training seeds. The same targets and checkpoints are used at every length; no $N = 0$ result is shown.

Model	$\text{Acc}_0, N = 1$	$\text{Acc}_1, N = 1$	$\text{Acc}_0, N = 200$	$\text{Acc}_1, N = 200$
Passthrough ⁸	85.26	72.03	85.26	72.03
Shandess Soft Prompt h10 ¹	94.12	4.81	86.95	76.34
Shandess Soft Prompt h1 (50/50) ¹²	62.65	39.12	87.85	75.03
Shandess Soft Prompt h1 (natural) ¹¹	68.73	32.11	87.45	75.17
Shandess HiCA h10 ²	52.49	76.03	87.55	75.70
Shandess HiCA h1 ³	74.90	74.37	88.35	74.23

Table 43: Conditional accuracy (%) at the endpoints of the fixed-target sweep. Every column uses the same 1,004 failures or 2,996 successes. Very short histories produce a strong failure-prediction bias for the Shandess Soft Prompt h10.

The conditional scores identify the Shandess Soft Prompt h10 failure mode: at $N = 1$ it predicts failure on 94.93% of targets, rising to 96.93% at $N = 2$. The h1 adaptations reduce but do not eliminate this imbalance. Admitting short histories during adaptation thus does not by itself resolve this regime; these runs do not isolate its cause. Conversely, h10 models retain slightly higher interaction accuracy at $N = 200$. Training on short histories does not produce a uniformly better checkpoint.

⁸DDPshandess2/PASSTHROUGH ¹gamma6/SOFT ¹²gamma1111/SOFT ¹¹gamma111/SOFT ²gamma6/FLAMINGO ³gamma8/FLAMINGO

E.3 First encounters with test-only exercises

Test-only identities are (source, exercise_id) keys present in the test split and absent from the non-test table, which also supplies validation. Targets must be eligible first encounters with the exercise in the student’s test records; histories retain interactions with other exercises. Of 55 test-only identities, all from AM, 54 have admissible targets. All 66 targets are retained. The 4,000 seen controls are sampled without replacement from 649,119 eligible AM controls (seed 1), covering 1,621 exercises. All six checkpoints produce complete predictions for the same 4,066 targets.

Model	Seen MeanAcc	Unseen MeanAcc	Unseen accuracy
Passthrough ⁸	80.29	56.18	51.52
Shandess Soft Prompt h10 ¹	81.48	88.74	86.36
Shandess Soft Prompt h1 (50/50) ¹²	82.20	91.35	86.36
Shandess Soft Prompt h1 (natural) ¹¹	81.50	88.74	86.36
Shandess HiCA h10 ²	82.09	90.38	84.85
Shandess HiCA h1 ³	82.50	93.27	89.39

Table 44: KT exercise cold start (%). All targets are AM first encounters: 4,000 seen and 66 test-only targets. MeanAcc is $(\text{Acc}_0 + \text{Acc}_1)/2$. Populations are not difficulty-matched; seen/unseen differences are descriptive. No recurrent KT baseline or Transformer/metadata-variant results are available in these archives.

Model	Acc ₀	Acc ₁	Macro	AUC	Brier
Passthrough ⁸	64.29	48.08	49.38	0.567	0.261
Shandess Soft Prompt h10 ¹	92.86	84.62	87.35	0.939	0.120
Shandess Soft Prompt h1 (50/50) ¹²	100.00	82.69	87.35	0.946	0.126
Shandess Soft Prompt h1 (natural) ¹¹	92.86	84.62	87.35	0.945	0.118
Shandess HiCA h10 ²	100.00	80.77	85.19	0.953	0.119
Shandess HiCA h1 ³	100.00	86.54	89.81	0.949	0.105

Table 45: Test-only exercise results: Acc₀, Acc₁ and unweighted macro accuracy across exercises are percentages. There are 14 incorrect and 52 correct targets, from 54 exercises and 21 students. AUC and Brier are on the unit scale.

The tables provide complementary information: the seen/unseen comparison and the conditional, macro and probability metrics for unseen exercises. The groups are not matched for difficulty or student population; mean stored history is 180.74 for test-only targets versus 127.18 for controls. Higher scores on test-only targets therefore do not demonstrate a benefit of novelty. The 100% Acc₀ values concern only 14 failures, and no local KT baseline was evaluated on this cohort.

E.4 Uncertainty of the cold-start comparisons

The paired student-bootstrap procedure follows Appendix C, with 2,000 replicates and seed 20260910. Here the fixed-target intervals use 1,075 students, while exercise intervals use the 21 students with test-only targets. Percentile bounds are the 2.5th and 97.5th percentiles of the interaction-weighted accuracy difference, not the MeanAcc difference. These exploratory intervals are not corrected for multiple comparisons.

⁸DDPshandess2/PASSTHROUGH ¹gamma6/SOFT ¹²gamma1111/SOFT ¹¹gamma111/SOFT ²gamma6/FLAMINGO ³gamma8/FLAMINGO

Paired comparison	Δ accuracy	95% CI	Students
Shandess Soft Prompt h10: N200 - N5	+2.30	[+1.35, +3.29]	1,075
Shandess Soft Prompt h1 (50/50): N200 - N5	+3.28	[+2.27, +4.29]	1,075
Shandess HiCA h10: N200 - N5	+2.20	[+1.02, +3.44]	1,075
Shandess HiCA h1: N200 - N5	+4.43	[+3.42, +5.42]	1,075
Shandess HiCA h1 - h10: N1	+4.38	[+3.04, +5.71]	1,075
Shandess HiCA h1 - h10: N200	-0.90	[-1.60, -0.25]	1,075
Unseen: Shandess Soft Prompt h1 (50/50) - Passthrough	+34.85	[+18.42, +47.92]	21
Unseen: Shandess HiCA h1 - Passthrough	+37.88	[+23.21, +46.67]	21
Unseen: Shandess HiCA h1 - Shandess Soft Prompt h1 (50/50)	+3.03	[-4.00, +9.09]	21

Table 46: Exploratory paired accuracy differences, in percentage points. The 50/50 label denotes the AM/MIA training mixture. Percentile intervals use 2,000 whole-student bootstrap replicates (seed 20260910), with fixed checkpoints and no correction for multiple comparisons.

For test-only exercises, one student contributes 18 targets and two others eight each. Of the 54 exercises, 43 have one target, ten have two and one has three. Macro exercise accuracy therefore relies largely on single observations. Student resampling describes variation in the observed student sample, not across training seeds or new exercise populations. In particular, the two-prediction advantage of Shandess HiCA h1 over the balanced Shandess Soft Prompt h1 adaptation does not establish a reliable ranking.

F Task instructions and illustrative input

The following system instructions are transcribed from the repository’s `shandess/prompts/kt_binary.py` and `shandess/prompts/answer.py`. Their source snapshots accompany the data package. They specify the target behavior; the history representation depends on the mode. Normal serializes past attempts in text. Shandess Soft Prompt and Shandess HiCA provide them through HEc, while Passthrough supplies no past interactions.

F.1 Knowledge Tracing system instruction

You are a knowledge-tracing model for French primary and secondary school mathematics. You will see a chronological sequence of past exercise attempts from a single student, each annotated with the outcome (correct/incorrect). You must predict whether THIS student will answer the next exercise incorrectly or correctly. Reply with 0 for incorrect or 1 for correct. Output exactly one character: 0 or 1. No explanation and no other text.

F.2 Answer Prediction system instruction

You are a student-modeling assistant for French primary and secondary school mathematics. You see a chronological sequence of past exercise attempts from a single student. These past attempts may include multiple-choice and non-multiple-choice exercises. Each attempt shows the exercise content, its gameplay type, and the observed outcome. For multiple-choice attempts, it also shows which option index the student picked when available. You must predict which option index THIS student will pick on the next multiple-choice question. Options are 0-indexed in reading order (top-to-bottom, left-to-right). Predict the student’s behavior, not the correct answer — students often make characteristic mistakes. Reply with a single non-negative integer. Output only the number.

F.3 Schematic example

The example below illustrates the organization of a textual AP input. It is synthetic and omits repository-specific formatting, exercise metadata and serialization details; it is not an extracted student record.

```
Attempt 1/1:
Exercise: What is 2 + 3?
Options: 0 -> 4; 1 -> 5; 2 -> 6
Observed student choice: 0
Observed outcome: incorrect

Next attempt:
Exercise: What is 3 + 4?
Options: 0 -> 6; 1 -> 7; 2 -> 8
This question has 3 options indexed 0..2 (reading order).
Which option will THIS student pick?
Reply with one integer from {0, 1, 2}. Output only the number.
```

The requested label is the student’s choice on the target exercise, not a solution supplied by the model. The target’s observed answer and correctness are supervision and are not part of the predictive input.